Machine Learning for Kira IBP Reduction

Seed selection and elimination ordering

October 5, 2026

Abstract

Feynman integral calculations are an important component of multiloop amplitude calculations in quantum field theory. In Quantum Chromodynamics (QCD), multiloop amplitudes and their associated Feynman integrals arise in higher-order perturbative calculations and can become increasingly complex at higher loop orders. Recent progress has automated these calculations to some extent using the framework Kira, which organizes target integrals into integral families and sectors, generate algebraic relations among the integrals by using integration by parts (IBP) identities, and reduce the resulting system to expressions in terms of a finite set of master integrals. We aim to explore the application of System 1 machine learning models to accelerate the IBP reduction process by investigating if lightweight learned models can improve seed selection. These models can predict the most useful integrals from which to generate IBP relations while minimizing redundant calculations. We will also explore System 1 models for learning efficient reduction and elimination sequences to minimize the number of algebraic operations required to express target integrals in terms of master integrals. Because the machine learning models will serve as decision-makers while the underlying IBP reduction and verification remain exact, these models can improve computational efficiency without compromising the correctness of the resulting reductions.

1 Introduction

Feynman integrals arise in higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes. In QCD, these corrections involve quark and gluon loops and contribute to predictions for cross sections and decay rates. Calculating a loop contribution requires integration over the independent internal momenta that remain after imposing momentum conservation [1].

The scalar integrals that appear in an amplitude have the general form

$$\mathcal{J} = \left(\prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}\right) \frac{\mathcal{N}(\boldsymbol{\ell}, \boldsymbol{p})}{\prod_{i=1}^{n_{\mathrm{phys}}} D_i^{a_i}}, \qquad D_i = q_i^2 - m_i^2 + \mathrm{i}0. \tag{1}$$

Here L is the number of loops and $\boldsymbol{\ell} = (\ell_1, \ldots, \ell_L)$ is the list of independent loop momenta over which we integrate. The number of independent external momenta is E, and we keep these momenta, $\boldsymbol{p} = (p_1, \ldots, p_E)$, fixed during integration. The numerator $\mathcal{N}$ is a polynomial obtained from the spin and tensor algebra; we leave its mass dependence implicit in the notation. The number of physical denominators is n_{phys}, and each D_i corresponds to an internal particle line with momentum q_i and mass m_i. The positive integer a_i is the power of D_i in the denominator, and q_i is a fixed linear combination of loop and external momenta, with $q_i^2 = q_i \cdot q_i$ denoting the Lorentz scalar product.

The integration dimension is $d = 4 - 2\epsilon$, where ϵ is the regulator used to define divergent integrals, so the result depends on ϵ as well as the external momenta and masses. The factor $1/(\mathrm{i}\pi^{d/2})$ is our normalization convention for each integration measure, with $\mathrm{i}^2 = -1$. In each denominator, i0 is an infinitesimal positive imaginary term that specifies how to integrate past a propagator pole, where $q_i^2 = m_i^2$.

Higher loop orders produce collections of integrals with related numerators and denominators and algebraic identities can be used to express them in terms of an independent set of master integrals. Kira automates this reduction [2], after which the calculation requires the values of the master integrals. To explain how the reduction works and where machine learning (ML) could speed it up, we first define the notation used in this paper, then derive the identities and describe how Kira solves the resulting equations.

2 The mathematical form of Feynman integrals

To label the integrals related to Eq. (1), we specify their momentum dependence and write the numerator and denominator powers as a vector of integers. This vector identifies each integral as an unknown in the reduction equations.

2.1 Loop momenta, propagators and regularization

After choosing how the loop and external momenta flow through the internal lines, we write each physical denominator as

$$D_i = q_i^2 - m_i^2 + \mathrm{i}0, \qquad q_i = \sum_{k=1}^{L} \alpha_{ik} \ell_k + \sum_{\beta=1}^{E} \eta_{i\beta} p_\beta. \tag{2}$$

The constants α_{ik} and $\eta_{i\beta}$ specify which loop and external momenta make up q_i, with indices k, j for loop momenta and β, γ for external momenta. Squaring q_i gives the scalar products $\ell_k \cdot \ell_j$, $\ell_k \cdot p_\beta$ and $p_\beta \cdot p_\gamma$, and those containing loop momenta vary during integration. The

vector $\boldsymbol{x}$ contains the independent external scalar products and squared masses, which are the kinematic parameters we keep fixed during integration.

Loop integrals can diverge at large momenta or when massless momenta become small or parallel. Dimensional regularization defines these integrals at values of d for which they converge and extends the result by analytic continuation to $d = 4 - 2\epsilon$, using additional regulators where necessary. In an expansion around $\epsilon = 0$, the divergences appear as negative powers of ϵ, which renormalization and infrared cancellation in the observable address later in the calculation [1]. Having specified the regulated measure and momentum dependence, we can now use the powers of the numerator and denominator factors to label individual integrals.

2.2 A common notation for an integral family

We add auxiliary expressions to the physical denominators so that we can write every required loop-momentum scalar product in terms of the list $D_1, \ldots, D_N$ and the fixed parameters $\boldsymbol{x}$. Here N is the number of expressions in the list, and the auxiliary expressions allow us to represent numerator factors. Writing the numerator in terms of this list expresses $\mathcal{J}$ as a linear combination of integrals of the form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \prod_{i=1}^{N} D_i^{-a_i}, \qquad \int_\ell \equiv \prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}, \qquad \boldsymbol{a} = (a_1, \ldots, a_N) \in \mathbb{Z}^N. \tag{3}$$

Here $\int_\ell$ is shorthand for the loop integration measure and $\boldsymbol{a}$ is the vector of N integer powers, with $\mathbb{Z}$ denoting the set of integers. These powers can be positive, zero or negative, so we can write numerator and denominator factors in the equivalent form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \frac{\prod_{i:a_i<0} D_i^{-a_i}}{\prod_{i:a_i>0} D_i^{a_i}}. \tag{4}$$

A positive index places $D_i^{a_i}$ in the denominator, a negative index places $D_i^{-a_i}$ in the numerator, and a zero index removes the factor, with an empty product equal to one. Auxiliary expressions used only as numerators therefore have nonpositive indices. An irreducible scalar product is a numerator scalar product that cannot be written as a linear combination of the physical denominators and terms depending only on $\boldsymbol{x}$ [3].

An integral family contains the integrals obtained by varying $\boldsymbol{a}$ for a fixed list $D_1, \ldots, D_N$ and fixed kinematics. Within this family, the target list specifies the integrals that the amplitude requires, and grouping family members by their positive indices organizes them into sectors.

2.3 Sectors and the organization of related integrals

A sector is a set of integrals with the same positive-index positions, so they contain the same denominators. We assign a label to each sector using

$$\theta_i(\boldsymbol{a}) = \begin{cases} 1, & a_i > 0, \\ 0, & a_i \leq 0, \end{cases} \qquad S(\boldsymbol{a}) = \sum_{i=1}^{N} 2^{i-1} \theta_i(\boldsymbol{a}). \tag{5}$$

The indicator θ_i is one when denominator i is present and zero otherwise, and S is an integer that identifies this set of denominators. Integrals with different powers of the same denominators belong to the same sector because S depends only on which indices are positive. Removing one or more denominators gives a subsector containing a proper subset of the original sector's denominators.

Scaleless sectors contain no nonzero mass or kinematic scale and therefore vanish in dimensional regularization, while changes of loop integration variables can yield symmetry relations between integrals. Kira uses these zero sectors and symmetry mappings to reduce the number of unknowns [2]. We can then use the sector and index labels to write the identities that relate the remaining family members.

3 From integration to algebraic reduction

To express the targets in terms of master integrals, we generate relations between family members through integration by parts and solve the resulting equations. Evaluating the master integrals then gives the target integral values needed to calculate the amplitude.

3.1 How integration by parts produces relations

For differentiable functions $f(z)$ and $g(z)$ of a real variable z, the product rule gives the integration identity between endpoints z_0, z_1:

$$\int_{z_0}^{z_1} f'(z)g(z)\,\mathrm{d}z = [f(z)g(z)]_{z_0}^{z_1} - \int_{z_0}^{z_1} f(z)g'(z)\,\mathrm{d}z. \tag{6}$$

Primes denote derivatives with respect to z, and the brackets denote the endpoint difference $f(z_1)g(z_1) - f(z_0)g(z_0)$. To apply this product rule to loop-momentum derivatives, we derive the integration-by-parts (IBP) identity where the regulated integral has a vanishing boundary term and then extend it by analytic continuation [4, 3]. For a loop momentum ℓ_k and a vector $v \in \{\ell_1, \ldots, \ell_L, p_1, \ldots, p_E\}$, the identity is

$$0 = \int_\ell \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_{i=1}^{N} D_i^{-a_i} \right). \tag{7}$$

The index μ is a spacetime index, and a repeated Lorentz index means that we sum over components using the spacetime metric. The derivative acts on both v^μ and the factors $D_i^{-a_i}$, so the product rule gives

$$0 = \int_\ell \left[\frac{\partial v^\mu}{\partial \ell_k^\mu} - \sum_{i=1}^{N} a_i \frac{v^\mu (\partial D_i / \partial \ell_k^\mu)}{D_i} \right] \prod_{j=1}^{N} D_j^{-a_j}. \tag{8}$$

The first term equals d for $v = \ell_k$ and vanishes when v is an external momentum or another independent loop momentum. Differentiating an inverse propagator produces scalar products, which we express through the family basis to convert each term into an integral with shifted indices:

$$\sum_{j=1}^{n_\lambda} c_{\lambda j}(\boldsymbol{a}; \boldsymbol{x}, \epsilon) I_{\boldsymbol{a}+\boldsymbol{\Delta}_{\lambda j}}(\boldsymbol{x}, \epsilon) = 0. \tag{9}$$

Here λ identifies the chosen pair (k, v), n_λ is the number of terms, and $\boldsymbol{\Delta}_{\lambda j}$ is the integer change in the index vector for term j. Differentiation gives the coefficient $c_{\lambda j}$, which depends on the integral indices and the parameters $\boldsymbol{x}, \epsilon$ [4].

The L choices of differentiated momentum and $L+E$ choices of v give $L(L+E)$ basic identities at each index vector, some of which may depend linearly on the others. Further relations follow from Lorentz invariance because a common rotation or Lorentz boost of all external momenta preserves a scalar integral, giving the infinitesimal identity

$$\sum_{\beta=1}^{E} \left(p_\beta^\nu \frac{\partial}{\partial p_{\beta\mu}} - p_\beta^\mu \frac{\partial}{\partial p_{\beta\nu}} \right) I_{\boldsymbol{a}} = 0. \tag{10}$$

Here μ, ν are free spacetime indices, meaning that the identity holds for each pair of components, and the derivatives act on the external-momentum dependence of the integral. Contracting these indices with external momenta gives scalar relations [5] that join the IBP and symmetry identities to form the reduction system.

3.2 From relations to master integrals

Let n_I be the number of distinct integrals in the generated equations and n_{eq} the number of equations. We arrange these integrals in a column vector so that the system takes the matrix form

$$A(\boldsymbol{x}, \epsilon)\, \boldsymbol{I} = 0, \qquad \boldsymbol{I} = (I_{\boldsymbol{a}^{(1)}}, \ldots, I_{\boldsymbol{a}^{(n_I)}})^{\mathsf{T}}. \tag{11}$$

The superscript T denotes transpose, so $\boldsymbol{I}$ is a column vector of integrals whose dependence on $\boldsymbol{x}, \epsilon$ we leave implicit here. The matrix A has n_{eq} rows and n_I columns, with each entry A_{ij} giving the coefficient of integral j in equation i. Solving this system expresses dependent integrals in terms of an independent set. This set is a master basis if the reduction relations express every required integral as a linear combination of its members. For the standard class of Feynman integrals at fixed graph, this basis contains a finite number of masters [6]. We write them as $M_1, \ldots, M_{n_M}$, where n_M is the number of master integrals, so the reduction formula is

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \sum_{m=1}^{n_M} C_{\boldsymbol{a}m}(\boldsymbol{x}, \epsilon) M_m(\boldsymbol{x}, \epsilon). \tag{12}$$

Here $C_{\boldsymbol{a}m}$ is the coefficient of master M_m in the reduction of target $I_{\boldsymbol{a}}$. The solver obtains it by adding, multiplying and dividing the identity coefficients, so it is a rational function, meaning a ratio of polynomials in $\boldsymbol{x}, \epsilon$. Different independent sets can define different master bases, and an insufficient equation set can leave additional integrals unresolved. Establishing the master basis therefore requires checking that the generated equations are enough to reduce the required integrals.

Equation (12) gives the coefficients multiplying the masters, whose evaluation requires a further calculation, for example by solving differential equations with known values or limits as boundary conditions [5]. Kira provides the reduction rules needed for this calculation by generating and processing the identities through the procedure described next.

4 How Kira performs the reduction

Kira implements the Laporta approach by generating a finite set of identities and eliminating integrals according to an ordering [7]. To obtain the coefficients in Eq. (12), Kira selects seeds, removes redundant equations and solves the retained system. We examine how Kira performs these steps to specify the seed and elimination choices that the proposed models would make.

4.1 Kira's input and seed generation

The user supplies the families in Eq. (3), relations among the parameters $\boldsymbol{x}$, the target list, and bounds on equation generation so that Kira can initialize the families and sectors. During this initialization, Kira detects scaleless sectors and identifies symmetry relations and mappings between compatible families [2].

A seed specifies the index vector $\boldsymbol{a}$ at which Kira evaluates Eq. (9) to generate several derivative identities, and the seed set may include integrals beyond the target list. The shifts $\boldsymbol{\Delta}_{\lambda j}$ in these

identities can introduce further integrals outside the seed set, so the resulting system depends on both the chosen seeds and the terms generated from them. To set limits on the indices of the seeds, Kira uses the quantities

$$t(\boldsymbol{a}) = \sum_{i=1}^{N} \theta_i(\boldsymbol{a}), \qquad r(\boldsymbol{a}) = \sum_{a_i>0} a_i, \tag{13}$$

$$s(\boldsymbol{a}) = -\sum_{a_i<0} a_i, \qquad n_{\mathrm{dot}}(\boldsymbol{a}) = r(\boldsymbol{a}) - t(\boldsymbol{a}). \tag{14}$$

Here t is the number of distinct denominators, r is the sum of their powers, and s is the sum of the numerator powers. The quantity n_{dot} is the sum of the extra denominator powers beyond one for each denominator. We use this symbol to keep d reserved for the integration dimension, and we restrict equation generation by specifying the allowed sectors and limits

$$r \le r_{\max}, \qquad s \le s_{\max}, \qquad n_{\mathrm{dot}} \le n_{\mathrm{dot,max}}, \tag{15}$$

where $r_{\max}$, $s_{\max}$ and $n_{\mathrm{dot,max}}$ are the upper limits on the corresponding quantities. The user sets $r_{\max}$ and $s_{\max}$ and may also set $n_{\mathrm{dot,max}}$. Because these choices determine which seeds contribute equations, a reduction run must establish whether the bounds supply enough relations for the targets [8].

Kira 3 changes seeding in symmetry-related sectors and provides the optional `truncate_sp` setting to restrict numerator seeding in subsectors, with parameter choices that require checks against the expected master basis [9]. The generated system can still contain redundant equations, which elimination identifies as it processes the relations into reduction rules.

4.2 Integral ordering, sector order and forward elimination

Elimination solves an identity for one integral and substitutes the result into other identities, using an integral ordering to select which unknown to eliminate from each equation. Kira defines these orderings through sector labels and the powers of the numerator and denominator factors, with choices that prioritize numerator powers or repeated denominators and thereby affect the master basis and runtime [8].

To describe this procedure, write $I_j = I_{\boldsymbol{a}^{(j)}}$ for the integrals in Eq. (11) and choose the ordering $I_1 \succ I_2 \succ \cdots \succ I_{n_I}$, where $\succ$ gives priority for elimination. In a row whose first nonzero term contains I_k, let b_j denote the coefficient of I_j, so choosing the nonzero coefficient b_k as the pivot gives

$$b_k I_k + \sum_{j>k} b_j I_j = 0 \quad \Longrightarrow \quad I_k = -\sum_{j>k} \frac{b_j}{b_k} I_j. \tag{16}$$

The coefficients b_j depend on $\boldsymbol{x}, \epsilon$, and substituting this expression into the other rows removes their dependence on I_k. Repeating this step gives forward elimination, which builds a triangular system in which each pivot integral depends on integrals later in the ordering. Back substitution then replaces the remaining pivot integrals using their own rules, completing the Laporta solution [7].

Sector scheduling sets the order in which the solver processes groups of integrals or equations, taking into account which other integrals their equations contain. Ordinary IBP identities for a sector involve that sector and its subsectors because nonpositive indices remain nonpositive when applying the derivative. Equations for one sector can therefore depend on integrals from its subsectors. The Kira 3 description starts equation generation with sectors earlier in its ordering and inserts previously obtained pivot relations into new equations [9], with the order of processing depending on the stage of the reduction.

When several rows share the same first integral, the choice of pivot row changes the substitutions that follow. Choosing row ρ to eliminate column k, for example, updates row i by

$$A_{ij} \longleftarrow A_{ij} - \frac{A_{ik}}{A_{\rho k}} A_{\rho j}, \qquad A_{\rho k} \neq 0. \tag{17}$$

Here i is the index of a row other than the pivot row, so $i \neq \rho$, and j is the column index. Subtracting the stated multiple of row ρ sets the coefficient in column k to zero and can create nonzero entries in other columns, a change called fill-in when a previously zero entry becomes nonzero. These substitutions also make the coefficient expressions longer by combining their numerators and denominators, so elimination can require more time and memory as it proceeds. Redundant equations add to this work [10], so Kira uses modular elimination to select equations before processing their full symbolic coefficients.

4.3 Removing redundant equations and selecting target dependencies

To reduce the number of equations it must solve, Kira identifies rows that follow from other equations. Elimination reduces every coefficient in such a row to zero, allowing the solver to omit that row. Kira tests these dependencies with modular arithmetic [2] by substituting sample values $\boldsymbol{x}_0, \epsilon_0$ and mapping the coefficients to a finite field $\mathbb{F}_P$ for a prime integer P:

$$A(\boldsymbol{x}, \epsilon) \longmapsto A(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{18}$$

Integers from 0 to $P - 1$ represent the elements of $\mathbb{F}_P$, and addition and multiplication keep results in this range by taking the remainder upon division by P. Division uses the modular inverse of a nonzero element, which multiplies that element to one modulo P, so the sampling procedure excludes points where a required denominator vanishes. This arithmetic keeps field elements bounded in size while allowing the solver to identify independent rows and retain their original symbolic identities.

The matrix rank is the largest number of independent rows, where no row is a linear combination of the others in the chosen set. Some sample values can make otherwise independent rows dependent, giving a rank smaller than that of the symbolic matrix. Checks must account for this possibility when using the probabilistic equation-selection method developed by Kant [10].

Kira also tracks which independent equations contribute to the target reductions, using the dependencies of the forward-eliminated system for this selection in Kira 3 [9]. Seed selection therefore supplies the integrals from which Kira generates equations, and equation selection then retains the relations needed for the targets so that the solver can perform the full reduction on this smaller system.

4.4 Back substitution and the final symbolic result

To obtain $C_{\boldsymbol{a}m}$ from the retained equations, Kira applies back substitution to the pivot rules produced by forward elimination. Since these rules can contain other pivot integrals, back substitution inserts their expressions recursively until the right-hand sides contain only masters, giving Eq. (12). Kira's original symbolic solver carries out this calculation using rational-function arithmetic through the computer algebra system Fermat [2].

Kira also solves systems at finite-field sample points and reconstructs the rational coefficients with FireFly, the reconstruction library introduced into the solver in Kira 2.0 [11]. In this procedure, each sampled coefficient has the form

$$C_{\boldsymbol{a}m}(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{19}$$

The modular operations use exact arithmetic, and reconstruction recovers $C_{\boldsymbol{am}}(\boldsymbol{x}, \epsilon)$ from sufficient samples before checking the result at additional points. Reconstruction therefore forms part of the calculation of the symbolic reduction coefficients, after which the master integrals still require evaluation [11].

Because seed selection determines the equations that enter this procedure and elimination decisions determine the substitutions and intermediate expressions, the proposed ML models would guide these two operations to reduce the time and memory needed for the reduction.

5 ML for seed selection and elimination ordering

To reduce the time and memory needed for IBP reduction, we will study two System 1 models: one for seed selection and one for elimination ordering. Here a System 1 model is a small machine learning model that learns from recorded reductions and makes a choice from the information available at a given stage. We would add the seed model before equation generation and the elimination model at the stage where the solver selects a pivot. Each model would assign scores to the available choices, and Kira would perform the algebra for the selected choice. Training these models would require target integrals with known master expansions, the seeds that supply their reduction equations, and the steps used to solve those equations. Section 7 describes the implemented seed model and its completed training, while Section 8 gives the held-out test results. Section 9 describes the implemented next-step model for reduction propagation, and Section 10 gives its held-out results.

5.1 Selecting seeds before equation generation

To select seeds, the first model would receive the family definition, the target index vectors $\boldsymbol{a}$, the chosen master basis and a set of candidate seeds within specified sectors and index bounds. For each candidate, it would use the seed indices and the target information to estimate how useful the resulting equations would be for the requested reduction.

During training, we would pair each target with the seed sets recorded in its known reductions. These examples would teach the model to rank seeds that together provide enough equations to reduce the target. Different reductions can use different sufficient seed sets, so the training data would retain these alternatives and their equation counts. When applying the trained model, we would select highly ranked seeds and ask Kira to generate their identities. If the equations leave a target unresolved in the chosen master basis, we would add further seeds and repeat the reduction. The second model would guide the elimination of the resulting equations.

5.2 Choosing elimination steps within the solver

To limit fill-in and the growth of coefficient expressions, the second model would rank the allowed pivot choices using the current equations, their nonzero coefficients and the integral ordering. For each recorded reduction step, the training example would contain the equations before the choice, the available pivots, the selected row and integral, and the operations that followed. The remaining recorded steps and their operation counts would show how that choice contributed to a complete reduction.

During a reduction, the model would select an allowed pivot and Kira would carry out the row operations in Eq. (17). The model would then receive the updated system and select the next pivot, continuing until Kira can complete the reduction by back substitution. Each choice must respect the integral ordering and chosen master basis. Preparing the model inputs and obtaining

its scores must take less time than its choices save in the algebraic calculation. Kira would use its existing reconstruction procedure for calculations over finite fields.

5.3 Integral data required for training and testing

To train and test these models, we require examples that specify the family denominators, kinematic parameters, targets and master basis, together with the exact coefficients in Eq. (12). For a target containing several integrals, these coefficients specify the master expansion of the complete expression. Each example must also identify the source equations, their seeds and derivative operators, and any symmetry relations used, so the seed model's choices determine the equations available to the elimination model.

We have constructed a collection with these data and retained the operations used both to form each target from masters and to reduce it back to the same basis. The elimination records specify the target's coefficients before and after each step, the available source rows, the selected pivot and the operation counts. These records describe elimination of a target coefficient row against a fixed set of source identities. Using the model within Kira will also require inputs and allowed choices that match Kira's evolving equation system.

The exact master coefficients supply the reference answer for checking complete reduction in the chosen basis, as discussed in Section 3 [9]. We keep these answers and construction histories separate from the problem information supplied to the models. Section 6 explains how we obtained the master bases, constructed the targets and checked the associated seed and elimination records.

6 Target Integral Generation for Machine Learning

To train and test the two models described in Section 5, we constructed targets from master bases while retaining their source identities, seeds and algebraic steps. The initial collection contains 49 master entries and 1,662 targets across 16 families. We then extended the collection to 1,000 families, giving 9,678 master entries and 240,472 targets with checked reference reductions [12, 13, 14]. The following subsections define the initial families and construction method, then explain the additional mass and kinematic conditions and the resulting data. All integrals remain symbolic throughout.

6.1 Initial one-loop families and their kinematics

We use the integral definition and normalization in Eq. (3), with $d = 4 - 2\epsilon$ and inverse propagators $D_i = q_i^2 - m_i^2 + \mathrm{i}0$. For each family, we specify the internal momenta q_i, the masses m_i and the scalar products of the external momenta, so every index vector defines a particular integral. In the equations below we suppress the common i0 prescription. We write $\mu = m^2$ for a common nonzero squared mass and $\mu_1 = m_1^2$, $\mu_2 = m_2^2$ for two independent nonzero squared masses. The symbols x_s and x_t denote momentum invariants and remain distinct from the index sums s and t in Eq. (14).

The one-loop collection begins with a massive tadpole, whose single inverse propagator is $D_1 = \ell_1^2 - \mu$ and which has no external momentum. We then include four bubble families with external momentum p and inverse propagators

$$D_1 = \ell_1^2 - m_1^2, \qquad D_2 = (\ell_1 - p)^2 - m_2^2, \qquad p^2 = x_s. \tag{20}$$

The squared-mass pairs are $(0,0)$, $(\mu,0)$, (μ,μ) and (μ_1,μ_2), corresponding to a massless bubble, a bubble with one massive line, an equal-mass bubble and an unequal-mass bubble. We keep x_s as an independent nonzero scale, with the massive families defined at generic values of their independent parameters.

To include three-point integrals, we define triangle families by

$$\begin{aligned} &D_1 = \ell_1^2 - m_1^2, \quad D_2 = (\ell_1 - p_1)^2 - m_2^2, \qquad D_3 = (\ell_1 - p_1 - p_2)^2 - m_3^2, \\ &p_1^2 = u_1, \qquad\quad p_2^2 = u_2, \qquad\quad (p_1+p_2)^2 = u_3. \end{aligned} \tag{21}$$

Here u_1, u_2, u_3 are the squared external momenta, also called external virtualities, and $p_1 \cdot p_2 = (u_3 - u_1 - u_2)/2$ fixes the remaining external scalar product. We use three massless families with $(u_1,u_2,u_3) = (0,0,x_s)$, $(0,x_2,x_3)$ and (x_1,x_2,x_3). The parameters x_1, x_2, x_3 denote independent nonzero virtualities. We also use two families with $m_1^2 = m_2^2 = m_3^2 = \mu$ and virtualities $(0,0,x_s)$ and (x_1,x_2,x_3). Thus the five triangle families vary both the internal masses and the number of nonzero external virtualities.

For four-point integrals, we use three massless box families with $p_4 = -p_1 - p_2 - p_3$ and

$$\begin{aligned} &D_1 = \ell_1^2, \qquad\quad D_2 = (\ell_1 - p_1)^2, \\ &D_3 = (\ell_1 - p_1 - p_2)^2, \quad D_4 = (\ell_1 - p_1 - p_2 - p_3)^2, \\ &x_s = (p_1+p_2)^2, \qquad x_t = (p_2+p_3)^2. \end{aligned} \tag{22}$$

Writing $u_i = p_i^2$ for $i = 1, \ldots, 4$, we fix the independent external scalar products through

$$p_1 \cdot p_2 = \frac{x_s - u_1 - u_2}{2}, \qquad p_2 \cdot p_3 = \frac{x_t - u_2 - u_3}{2}, \qquad p_1 \cdot p_3 = \frac{u_4 + u_2 - x_s - x_t}{2}. \tag{23}$$

The virtuality choices are $(0,0,0,0)$, $(0,0,0,x_4)$ and (x_1,x_2,x_3,x_4), where x_4 is the fourth nonzero virtuality parameter. These choices give boxes with zero, one and four off-shell legs; in this classification an on-shell external leg has $p_i^2 = 0$. We treat the remaining invariants as independent subject to the stated relations. These definitions give 13 one-loop families, which we extend with the following three two-loop families.

6.2 Initial two-loop families and auxiliary expressions

The two-loop families use loop momenta ℓ_1, ℓ_2. For a massless two-point integral with external momentum p and $p^2 = x_s$, we take

$$(D_1, D_2, D_3, D_4, D_5) = (\ell_1^2, \ell_2^2, (\ell_1 - \ell_2)^2, (\ell_1 - p)^2, (\ell_2 - p)^2). \tag{24}$$

These five expressions determine the five loop-dependent scalar products $\ell_1^2, \ell_2^2, \ell_1 \cdot \ell_2, \ell_1 \cdot p$ and $\ell_2 \cdot p$. Varying their indices therefore includes numerator insertions as well as repeated propagators.

For the equal-mass sunrise with the same external invariant, we use

$$\begin{aligned} &D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (p - \ell_1 - \ell_2)^2 - \mu, \\ &D_4 = (p - \ell_1)^2, \quad D_5 = (p - \ell_2)^2, \quad a_4, a_5 \le 0. \end{aligned} \tag{25}$$

The first three expressions are the physical inverse propagators. The auxiliary expressions D_4, D_5 complete the five scalar products needed to express the derivatives in the IBP identities. Their nonpositive indices restrict them to numerator factors or their absence, so the top sector contains only D_1, D_2, D_3 as denominators. We keep x_s and μ independent.

The final family is the equal-mass two-loop vacuum sunset, which has no external momentum and uses

$$D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (\ell_1 - \ell_2)^2 - \mu. \tag{26}$$

These three expressions determine all loop scalar products for this vacuum family. For every family, we include the sector containing all physical propagators and all its subsectors, so the resulting basis also accounts for integrals with removed propagators. The number of physical propagators in this top sector is denoted by t_{top}. It is one, two, three and four for the tadpole, bubble, triangle and box families, respectively, five for the massless two-loop propagator, and three for the sunrise and vacuum families. This number sets the initial range of identities used to identify the masters.

6.3 Identification and checks of the master bases

For each initial family, we used Kira 3.1 to generate IBP relations together with the applicable Lorentz-invariance and symmetry relations. Kira removes scaleless sectors and uses finite-field elimination to identify the integrals that remain independent in the equations needed for a selected integral range [2, 9]. We used Kira's integral ordering 1 throughout, which orders sectors by their labels and prefers numerator insertions over extra denominator powers when selecting representatives within a sector [8]. Keeping this ordering fixed gives a specified basis against which to compare later calculations.

We first generated identities from seeds satisfying

$$r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2, \tag{27}$$

and selected integrals to probe the basis within

$$r \le t_{\text{top}} + 1, \qquad s \le 1, \qquad n_{\text{dot}} \le 1. \tag{28}$$

Here r is the sum of positive indices, s is minus the sum of negative indices and $n_{\text{dot}} = r - t$ is the number of extra denominator powers, as defined in Eq. (14). Both sets of limits apply to the top sector and its subsectors. The probe integrals specify the range for which Kira must identify the required masters, while the wider seed range supplies relations that can eliminate the additional integrals introduced by index shifts. These probes serve the basis calculation; the ML examples come from the subsequent reverse construction.

Kira evaluated the coefficients of these relations over a finite field using the substitution in Eq. (18). For the first calculation we used the prime $P_1 = 2147483647$ and dimension value $d_1 = 37$. To specify the kinematic substitutions, define the ordered parameter list

$$\boldsymbol{z} = (x_s, x_t, x_1, x_2, x_3, x_4, \mu, \mu_1, \mu_2). \tag{29}$$

The first sample was

$$\boldsymbol{z}^{(1)} = (-13, -19, -11, -17, -29, -43, 7, 7, 23), \tag{30}$$

using only the entries present in each family and retaining all zeros and equal-mass conditions in its definition. We interpret these assignments modulo P_1 when evaluating the identity coefficients. The dimension sample corresponds to $\epsilon_1 = (4 - d_1)/2$ in the same field. These substitutions test algebraic relations; the master labels continue to refer to integrals at symbolic $\boldsymbol{x}, \epsilon$.

We then enlarged the seed and probe ranges to check whether the initial basis remained sufficient:

$$\begin{aligned} &\text{seeds:} \quad r \le t_{\text{top}} + 3, \qquad s \le 3, \qquad n_{\text{dot}} \le 3, \\ &\text{probes:} \quad r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2. \end{aligned} \tag{31}$$

We repeated the calculation first at the original sample and then at a second sample with prime $P_2 = 1000000007$, dimension $d_2 = 41$ and

$$\boldsymbol{z}^{(2)} = (-31, -47, -23, -37, -53, -71, 11, 11, 29). \tag{32}$$

Changing the prime, the dimension and the independent kinematic parameters provides a second check of the linear dependencies, since particular substitutions can lower the rank of the symbolic coefficient matrix [10].

In both enlarged calculations, we supplied the original masters as the preferred basis and enabled Kira's check for additional or unused masters [9]. We required the complete list of master index vectors to agree in all three calculations, and also checked each vector against its family definition and sector label. All 16 families satisfied these requirements, giving 48 completed master-identification calculations. The agreement provides evidence for basis stability over the tested index ranges and sample points. Completeness for arbitrary integer indices would require further relations or an independent argument, and special kinematic limits can change the basis.

We stopped each calculation after master identification. This stage determines which symbolic integrals form the starting basis within the checked domains. We then obtained rational reduction coefficients for the constructed targets through the algebraic procedure described below. The numerical values and ϵ expansions of the master integrals remain separate from this basis determination.

6.4 Resulting bases and the starting coefficients

The massive tadpole gave one master. The massless, one-mass, equal-mass and unequal-mass bubbles gave one, two, two and three masters, respectively. The massive bubble bases include the necessary tadpole subsectors, with two distinct tadpoles when the internal masses differ.

The massless triangles with one, two and three nonzero external virtualities gave one, two and four masters, respectively. For the first two families, Kira chose bubble integrals in subsectors as the masters. The three-virtuality family also retained a triangle integral. The equal-mass triangles with one and three nonzero virtualities gave three and five masters. The massless boxes with zero, one and four off-shell legs gave three, four and eleven masters, respectively, including the required lower sectors in each count.

The massless two-loop propagator gave two masters, the equal-mass sunrise gave three, and the equal-mass vacuum sunset gave two. For the denominator order in Eq. (25), the sunrise basis is

$$M_1 = I_{(1,1,0,0,0)}, \qquad M_2 = I_{(1,1,1,0,0)}, \qquad M_3 = I_{(1,1,1,-2,0)}. \tag{33}$$

The first integral factorizes into a product of two massive tadpoles because removing D_3 separates the loop integrations. The other two integrals retain all three sunrise propagators, and M_3 contains the numerator factor D_4^2. This example shows how the chosen ordering can retain a numerator integral as a master.

The counts sum to 49 master entries across these 16 initial families. We count each basis within its own family, so shared tadpole and bubble subsectors can appear in more than one family. The complete family definitions and master index vectors are available with the accompanying data [12]. Each entry specifies the denominators, kinematics, index vector, sector and mass dimension. With the normalization in Eq. (3), the latter is

$$[I_{\boldsymbol{a}}]_{\text{mass}} = Ld - 2\sum_{i=1}^{N} a_i, \tag{34}$$

where $[I_{\boldsymbol{a}}]_{\text{mass}}$ is the power of mass carried by the integral. Each loop measure contributes d powers and each factor $D_i^{-a_i}$ contributes $-2a_i$. This dimension constrains the coefficients used to combine integrals in the target construction.

For that construction, we kept the identified masters symbolic and initialized each master as itself. If $\boldsymbol{\mu}^{(m)}$ is the index vector of master M_m, the starting coefficients are

$$M_m = I_{\boldsymbol{\mu}^{(m)}}, \qquad C_{\boldsymbol{\mu}^{(m)}n} = \delta_{mn}, \qquad m, n = 1, \ldots, n_M. \tag{35}$$

Here n_M is the number of masters in the selected family, and δ_{mn} is one when $m = n$ and zero otherwise. Thus each master has coefficient one for itself and zero for the others. We held the family definition and basis fixed while constructing its targets, starting with these masters and extending the collection through the identities below. Each accepted target therefore has a specified reference basis and an exact master expansion.

6.5 Source identities and the reference integral order

To extend the starting masters into targets, we first generated relations among the integrals of each family. We used the IBP identity in Eq. (7) with seeds in every nonempty physical sector within the bounds of Eq. (27). The auxiliary sunrise indices remained nonpositive. We fixed this candidate seed set before constructing targets, and retained each equation with its seed $\boldsymbol{\sigma}$, differentiated loop momentum ℓ_k and vector v [4, 2]. The resulting equations can contain integrals beyond the seed bounds because differentiation shifts the indices.

The vectors v included loop momenta, external momenta and propagator momenta q_i. Each propagator momentum is a fixed linear combination of loop and external momenta, so its IBP equation is the corresponding linear combination of the usual IBP equations at the same seed. We retained this decomposition to specify how each equation follows from its source operators. We also obtained symmetry relations by explicit changes of loop variables whose loop transformation matrix has determinant $+1$ or -1. We accepted a transformation when it permuted the inverse propagators with their masses unchanged and preserved the physical and auxiliary index positions.

To give the constructions a reduction order, we protected the chosen masters from elimination and ordered every other integral by

$$(S(\boldsymbol{a}),\, n_{\text{dot}}(\boldsymbol{a}),\, s(\boldsymbol{a}),\, r(\boldsymbol{a}),\, \boldsymbol{a}). \tag{36}$$

Here S is the sector number in Eq. (5), and the remaining quantities are the index sums in Eq. (14). We compare the entries in the displayed order, using the first unequal entry to decide which integral is larger. We compare the components of $\boldsymbol{a}$ in the same way when all earlier entries agree. The leading integral of an equation is its largest integral outside the master basis. This reference order specifies the construction and recorded reductions; adapting the elimination model to Kira will require its actual ordering and available pivot choices.

6.6 Reverse construction from combinations of masters

To obtain target expressions with known answers, we selected subsets of masters from the same family and replaced their terms through the source identities. Let $\mathcal{J} = \{m_1, \ldots, m_h\}$ be the selected master labels, where h is the number of chosen masters. The initial expression is

$$\mathcal{T}_0 = \sum_{m \in \mathcal{J}} c_m(\boldsymbol{x}, d) M_m, \qquad 1 \le h \le \min(3, n_M). \tag{37}$$

The coefficients are rational functions of the independent kinematic parameters and $d = 4 - 2\epsilon$, with no dependence on the loop momenta. Writing them in terms of d is equivalent to writing them in terms of ϵ. Their mass dimensions must compensate for differences between the chosen masters so that every term in $\mathcal{T}_0$ has the same dimension.

For this purpose, let $B_m = \sum_i \mu_i^{(m)}$ be the sum of the indices of master M_m, and set $A = B_{m_1}$. For a fixed nonzero squared mass or momentum invariant Q^2 in the family, we used

$$c_{m_j} = u_j(d)(Q^2)^{B_{m_j}-A}, \qquad [\mathcal{T}_0]_{\text{mass}} = Ld - 2A. \tag{38}$$

Here $j = 1, \ldots, h$ identifies a selected master and u_j is dimensionless. We varied the weights through the three forms

$$\begin{gathered} u_j(d) \in \left\{ \eta_j,\ \eta_j \frac{d-2}{d-4},\ \eta_j \frac{j+1}{d-2} \right\}, \\ \eta_j \in \{-3, -2, -1, 1, 2, 3\}. \end{gathered} \tag{39}$$

Here η_j is a nonzero integer weight. For each sequence we chose one of these three forms and varied the integer weights and selected masters. The power of Q^2 in Eq. (38) gives c_m mass dimension $2(B_m - A)$, so Eq. (34) gives the stated common dimension.

To replace a term, write the current expression and a source IBP identity as

$$\mathcal{T}_n = \sum_{\boldsymbol{b}} w_{\boldsymbol{b}}^{(n)} I_{\boldsymbol{b}}, \qquad R = \sum_{\boldsymbol{b}} r_{\boldsymbol{b}} I_{\boldsymbol{b}} = 0. \tag{40}$$

Here n is the number of completed replacements, while $w_{\boldsymbol{b}}^{(n)}$ and $r_{\boldsymbol{b}}$ are the coefficients of integral $I_{\boldsymbol{b}}$ in the expression and identity. The sums contain their finitely many nonzero terms. For a present term $I_{\boldsymbol{a}}$, we selected an identity generated at seed $\boldsymbol{a}$ with $r_{\boldsymbol{a}} \neq 0$, and formed

$$\mathcal{T}_{n+1} = \mathcal{T}_n - \frac{w_{\boldsymbol{a}}^{(n)}}{r_{\boldsymbol{a}}} R. \tag{41}$$

This operation cancels the chosen term and introduces other family integrals. Since $R = 0$, the resulting expression has the same master expansion as $\mathcal{T}_0$. We required the identity to introduce a leading integral larger than every integral already present, so the recorded steps can be reversed through the order in Eq. (36).

For the initial collection, we attempted 24 sequences per family, with at most four replacements per sequence, and retained accepted intermediate expressions. A sequence stopped when no admissible replacement was found within the construction limits. We removed identical target expressions and retained only nonzero targets with complete reference reductions. When the final expression contained one integral with coefficient $q \neq 0$, we divided the expression and its master coefficients by q and recorded this normalization. Expressions containing several index vectors remain targets for reduction of the complete sum. To obtain further individual integrals, we also applied the source identities in the following way.

6.7 Constructing individual integrals from known reductions

Starting from Eq. (35), we searched for a source identity whose leading integral was the only term without a known master expansion. Such an identity has the form

$$h_0 I_{\boldsymbol{b}} + \sum_{j=1}^{n_R} h_j I_{\boldsymbol{a}^{(j)}} = 0, \qquad h_0 \neq 0, \tag{42}$$

where $I_{\boldsymbol{b}}$ is the new integral, n_R is the number of other terms and h_0, h_j are the identity coefficients. Substituting the known reductions of the other terms gives

$$I_{\boldsymbol{b}} = \sum_{m=1}^{n_M} C_{\boldsymbol{b}m} M_m, \qquad C_{\boldsymbol{b}m} = -\sum_{j=1}^{n_R} \frac{h_j}{h_0} C_{\boldsymbol{a}^{(j)}m}. \tag{43}$$

We added each resulting expansion to the known set so later identities could use it. If all its master coefficients vanished, we retained the zero-integral proof as a dependency for subsequent constructions. The target collection contains the accepted nonzero examples.

For the initial collection, we made at most five passes through the source equations and allowed at most 64 new nonzero constructions of this type per family. An equation with several still-unknown integrals became usable only after other identities supplied the missing expansions. Solving remaining coupled systems would extend this construction beyond the individual integrals obtained here. Both this procedure and the reverse replacements above preserve the source equations and their dependencies, which determine the seed and elimination records.

6.8 Seed records and elimination sequences

For each accepted target, we retained its exact master coefficients together with the source identities used to obtain them. An IBP identity contributes its seed and derivative operator, while a symmetry identity contributes its loop-variable transformation and integral mapping. Dependencies carry the earlier identities needed to reduce intermediate integrals or establish that they vanish. For an IBP step of the form in Eq. (42), the sufficient seed set therefore obeys

$$\mathcal{S}_{\boldsymbol{b}} = \{\boldsymbol{\sigma}\} \cup \bigcup_{j=1}^{n_R} \mathcal{S}_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{S}_{\boldsymbol{\mu}^{(m)}} = \varnothing. \tag{44}$$

Here $\mathcal{S}_{\boldsymbol{a}}$ is the seed set recorded for $I_{\boldsymbol{a}}$, $\boldsymbol{\sigma}$ is the seed of the current IBP identity and $\varnothing$ is the empty set. Symmetry steps retain the dependency sets and their symmetry record. For reverse constructions, the seed set is the union of the seeds of the identities used along the recorded path.

We retained the full candidate seed set alongside each family so the seed model can compare possible choices. The recorded subset supplies enough equations for the known reduction, and seeds outside it can still contribute to other valid reductions. For the model inputs, we stored the family, basis, target coefficients and index information, including the sector and $t, r, s, n_{\mathrm{dot}}$. We kept the exact answer, construction history and reference seed choices separately.

To obtain elimination examples, we represented the target as a row of coefficients $T_{\boldsymbol{b}}$ multiplying $I_{\boldsymbol{b}}$. At each step, we selected its largest remaining integral outside the master basis, denoted by $I_{\boldsymbol{a}}$. The available pivot rows were the retained source identities with leading integral $I_{\boldsymbol{a}}$ and a nonzero coefficient for that integral. For the chosen row R, the update was

$$T_{\boldsymbol{b}} \longleftarrow T_{\boldsymbol{b}} - \frac{T_{\boldsymbol{a}}}{r_{\boldsymbol{a}}} r_{\boldsymbol{b}}. \tag{45}$$

This cancels $I_{\boldsymbol{a}}$ from the target and can introduce only smaller integrals or masters. Repeating the recorded choices gives the target's known master expansion. The source equations remain unchanged during this target-row reduction, so the current row, source equations and reference order specify the state seen by the elimination model.

For every step, we retained the row before and after the update, all available pivot rows, the selected equation and integral, the multiplier, and the numbers of new and cancelled nonzero entries. We kept the coefficients as exact rational functions and also stored numerical row states

over a finite field. The first sample in Eq. (30), with $P_1 = 2147483647$ and $d_1 = 37$, supplied these numerical states. Substituting parameters into the coefficient functions leaves the integrals symbolic and uses exact modular arithmetic for the row operations [11].

For a selected source row with n_{row} nonzero terms, we counted one finite-field division and n_{row} multiplications and subtractions. Summing these counts gives the cost of the recorded target-row sequence. Equation generation, forming combined IBP operators, symbolic simplification, model inference and reconstruction fall outside this count. The construction gives a valid reference path and an upper bound on the minimum cost under these rules. Establishing a minimum would require a separate argument covering alternative seed sets and elimination paths.

6.9 Initial target collection and consistency checks

The initial collection contains 1,662 accepted targets across its 16 families, with 309 individual integrals, 1,353 integral expressions and 5,138 recorded elimination steps [13]. We divided the examples by complete families into 1,290 training targets, 185 validation targets and 187 test targets. Validation targets belong to the unequal-mass bubble and massless triangle with three off-shell legs. Test targets belong to the massless box with four off-shell legs and the equal-mass sunrise. These held-out targets contain terms in the full physical sector, while their basis definitions still include the required lower sectors.

In this initial collection, the validation set consists entirely of integral expressions, and the test set contains one individual integral and 186 expressions. These initial counts motivate the additional held-out individual constructions described below. This initial collection covers the specified one-loop and two-loop families and the stated construction bounds. The following sections describe seed-model training and its held-out comparison on the expanded collection. Section 9 describes the reduction model. Section 10 measures complete target-row sequences and their selection costs. Measurements of full Kira jobs remain a later stage.

Before accepting the collection, we rederived all 55,654 stored source equations from their family definitions and checked the recorded symmetry transformations. We also checked 2,699 used IBP equations by independently differentiating the original quadratic expressions for the propagators in terms of loop scalar products. We then replayed every target construction and elimination sequence with exact rational functions, including intermediate zero-integral dependencies and final normalizations. Each final result agreed with its stored master expansion, and the checks also confirmed mass dimensions, seed records and allowed pivots.

To check the sampled coefficients, we repeated every elimination sequence at three finite-field points. The first two were the samples in Eqs. (30) and (32), with the primes and dimension values specified there. The third used $P_3 = 1000000009$, $d_3 = 43$ and

$$\boldsymbol{z}^{(3)} = (-59, -73, -41, -61, -83, -101, 17, 17, 47), \tag{46}$$

in the parameter order of Eq. (29). Each family used its own parameters and retained its prescribed zeros and equal masses. All required divisions were defined at the three points, and every sampled reduction reproduced the reference coefficients.

The exact symbolic checks establish the recorded reductions from their source identities at generic parameters, while the three-point checks verify the numerical data supplied with them. This construction already provides the symbolic coefficient functions, so obtaining the reference answers requires no rational-function reconstruction. The accompanying data contain the targets, equations, construction histories, sufficient seeds and elimination records needed to train and test the two proposed models [13].

6.10 Extension to different mass and kinematic conditions

To supply examples across a wider range of parameter conditions, we added 984 one-loop families with four physical propagators. For these families, we write

$$D_i = (\ell_1 - Q_i)^2 - \mu_{\alpha_i} + \mathrm{i}0, \qquad i = 1, \ldots, 4, \qquad Q_1 = 0. \tag{47}$$

Here Q_2, Q_3, Q_4 are three independent external momentum vectors, $\mu_0 = 0$ and μ_1, μ_2, μ_3 are independent nonzero squared masses. Each label $\alpha_i \in \{0, 1, 2, 3\}$ assigns the squared mass of the corresponding propagator. Repeated positive labels impose equal masses.

We specify the external kinematics through the six squared differences between the propagator shifts:

$$\Delta_{ij} = (Q_i - Q_j)^2 = y_{\beta_{ij}}, \qquad 1 \le i < j \le 4. \tag{48}$$

Here $y_0 = 0$, y_1, y_2, y_3 are independent nonzero momentum invariants, and $\beta_{ij} \in \{0, 1, 2, 3\}$. Repeated labels impose equality between the corresponding invariants. The mass parameters and momentum invariants remain independent of one another, and a family uses only the parameters that occur in its assignments.

These distances determine the external scalar products:

$$G^{\mathrm{ext}}_{ij} = Q_i \cdot Q_j = \frac{\Delta_{1i} + \Delta_{1j} - \Delta_{ij}}{2}, \qquad i, j = 2, 3, 4, \qquad \Delta_{ii} = 0. \tag{49}$$

The matrix G^{ext} is the external Gram matrix. We required it to be nonsingular at all three recorded sample points, which also excludes a determinant that vanishes identically under the stated parameter conditions. Each family then defines a symbolic integral problem through its particular zero and equality relations.

To remove equivalent definitions, we compared the assignments under all 24 permutations of the four propagators, together with independent relabelling of the nonzero mass and momentum-invariant parameters. The enumeration found 1,673 inequivalent profiles satisfying the parameter and sample requirements after excluding the existing massless box definitions. A fixed ordering selected 984 of these profiles [14]. Together with the original families, the collection contains 997 one-loop and three two-loop families. The added profiles share the four-propagator box topology, so the family count describes different mass and kinematic conditions within this topology.

We identified and checked a master basis separately for every added family using the three calculations in Subsection 6.3. The four-propagator families have $t_{\mathrm{top}} = 4$, so the first seed range has $r \le 6$, $s \le 2$ and $n_{\mathrm{dot}} \le 2$. We then enlarged the seed and probe ranges, supplied the first masters as the preferred basis and repeated the basis check at a second point and prime. All 1,000 families passed their three basis calculations, giving 3,000 completed checks within the recorded domains.

For the added families, define

$$\boldsymbol{w} = (y_1, y_2, y_3, \mu_1, \mu_2, \mu_3). \tag{50}$$

We used the same three primes and dimension values specified above, with the parameter assignments

$$\begin{aligned} \boldsymbol{w}^{(1)} &= (-11, -19, -29, 7, 13, 23), \\ \boldsymbol{w}^{(2)} &= (-31, -47, -61, 11, 17, 29), \\ \boldsymbol{w}^{(3)} &= (-41, -67, -83, 17, 23, 37). \end{aligned} \tag{51}$$

Each family uses only its present parameters and retains all specified zeros and equalities. The first two points supply the basis checks, while the target validation uses all three.

6.11 Expanded targets and their verified reductions

Using the added bases, we applied the reverse construction in Eq. (41) with 48 attempted sequences per family and at most four replacements per sequence. We also allowed up to 96 nonzero individual-integral constructions per added family, using at most five passes through the source identities. The initial master weights retain the rational functions and mass-dimension conditions in Eqs. (38) and (39).

When the first construction produced no accepted targets, we extended the candidate equations to every retained IBP identity containing the selected current integral. Its seed $\boldsymbol{\sigma}$ can then differ from that integral's index vector $\boldsymbol{a}$. The substitution still uses the coefficient of $I_{\boldsymbol{a}}$ in the selected identity and preserves the known master expansion. This extension allows a reverse path to enter a higher sector through an identity generated there, including cases whose masters all lie in lower sectors. We retain the actual seed, require the new leading integral to increase under the reference order, and check every step in the resulting reduction.

To add individual integrals to the held-out sets, we also used the checked construction proofs to select integrals whose four indices are all nonzero and include at least one negative index. Each selected integral already has a known master expansion. We retained it as a separate target and rebuilt its elimination trace from the recorded identities. This step added 384 individual targets. Each has one denominator factor and three numerator factors, with their powers specified by the index vector. The complete four-expression definition therefore enters the target, while its positive indices place it in a one-propagator sector.

The combined collection contains 9,678 master entries and 240,472 targets: 55,768 individual integrals and 184,704 integral expressions [14]. We count master entries within each family, so related lower-sector integrals can occur in several bases. Every target has its complete master expansion, source identities, sufficient seeds and recorded elimination path.

For the added families, we kept each complete kinematic zero and equality pattern in one data split across all mass assignments. This grouping also respects the original training box profiles. We assigned the original validation bubble and triangle examples to the combined training set because those integral families also occur as lower sectors of the added training families. The new grouped families supply the expanded validation set. The combined split contains 771 training families with 198,527 targets, 106 validation families with 19,600 targets, and 123 test families with 22,345 targets. The reverse sequences require validation and test expressions to contain terms in their full physical sectors. The additional individual targets instead require all four family expressions to occur, including those in the numerator. Shared lower-sector masters remain part of the family definitions. The complete family profile is held out, while some lower-sector reduction structure can also occur in training.

The validation set contains 183 individual integrals and 19,417 integral expressions, while the test set contains 202 individual integrals and 22,143 expressions. These counts specify the available tests for each target type. The added individual targets supply tests of numerator integrals in one-propagator sectors. Individual targets with more positive denominator indices and additional high-loop families require further constructions. Sections 8 and 10 measure the two models on the current grouped split.

We rederived 3,299,902 source equations and checked 227,330 used IBP equations by independent differentiation across the complete collection. We replayed all 240,472 target constructions and 832,924 recorded elimination steps with exact rational functions. Every reduction reproduced its reference master coefficients, and the three finite-field replays checked the corresponding numerical rows. We also checked family equivalence, the grouping of data splits and the correspondence between every saved target and its records. The accompanying collection provides the family definitions, bases, targets and complete reduction data [14].

7 Machine Learning Model for Seed Selection

To train and test the seed-selection model, we use the target expressions and sufficient seed sets constructed in Section 6. We have implemented a small neural network that assigns a binary decision and its estimated confidence to each supplied candidate seed. We trained the model on the fixed training division and then evaluated it on the held-out targets. This section defines the inputs, probabilities and training procedure, and Section 8 gives the resulting seed predictions.

7.1 Model inputs and seed probabilities

A seed specifies an integral index vector at which the solver generates IBP identities. The resulting equations can contain other integrals whose elimination uses identities from additional seed vectors [2]. A target can therefore use several seeds in a sufficient reduction set. Some targets admit a reduction from one seed, while a target already expressed in the chosen masters needs no IBP seed. We formulate the prediction as a separate binary decision for every candidate, so the model can select several seeds for the same target.

For a fixed family and chosen master basis, write a target as

$$\mathcal{T} = \sum_{j=1}^{n_T} c_j(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{C} = \{\boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{n_C}\}. \tag{52}$$

Here n_T is the number of terms in the target, c_j is the rational coefficient of its jth term, $\boldsymbol{x}$ is the set of kinematic parameters and d is the space-time dimension. The vector $\boldsymbol{a}^{(j)}$ gives the integral indices of that term. The set $\mathcal{C}$ contains n_C candidate seed vectors within the stored family bounds. It supplies all candidates that this implementation scores for the target. Extending the seed bounds would require supplying an enlarged candidate set.

The model receives every term in the stored order, including its indices and symbolic coefficient. Each integral contributes its index values, the locations of positive indices and the quantities r, s, t and n_{dot} defined in Eq. (14). We transform an index value a to $\mathrm{sgn}(a)\log(1+|a|)$ and transform the nonnegative counts with $\log(1+v)$, where v denotes the count. The input also records the number of family indices and which positions are present. The current representation has capacity for 16 indices; the existing families use at most five.

To include the coefficient functions, we separate their written expressions into variable names, integers and mathematical symbols. We encode counts of individual symbols and consecutive groups of two and three symbols in a fixed vector with 64 components. A fixed signed assignment maps each group to a component, and division by the square root of the total number of groups sets the scale. Every coefficient symbol enters this representation, although different groups can share a component. The resulting vector is a compressed description of the coefficient expression. It does not perform symbolic simplification or establish equality between different expressions. In particular, modular residues do not enter as ordinary real coefficient values.

For each term we append its relative position in the stored list and the inverse term count. Denote the resulting term vector by $\boldsymbol{f}_j$. A shared learned map ϕ_θ forms

$$\boldsymbol{h}_j = \phi_\theta(\boldsymbol{f}_j), \qquad \boldsymbol{h}_j \in \mathbb{R}^{32}. \tag{53}$$

Here θ denotes the adjustable parameters of the model and $\boldsymbol{h}_j$ is a vector describing the term. The same map acts on each term, so its number of parameters does not depend on n_T. Combining shared term maps through averages follows the set-based construction of Ref. [15], with additional position information to retain the stored order.

We also include every ordered pair of distinct terms. If $n_T > 1$, their combined description is

$$\boldsymbol{h}_{\text{pair}} = \frac{1}{n_T(n_T-1)} \sum_{\substack{j,k=1 \\ j\neq k}}^{n_T} \psi_\theta([\boldsymbol{h}_j, \boldsymbol{h}_k, \boldsymbol{h}_j - \boldsymbol{h}_k, \boldsymbol{h}_j \odot \boldsymbol{h}_k])\,. \tag{54}$$

Square brackets join vectors, $\odot$ multiplies corresponding components, and ψ_θ maps each joined vector to 32 components. For an individual-integral target, we set $\boldsymbol{h}_{\text{pair}} = \boldsymbol{0}$. This representation includes both orders of each pair and the positions of the associated terms. The present model uses pairs as its specified form of term combinations; it does not enumerate longer subsets or all permutations of the full expression. The fixed stored term order must also be used when preparing later inputs.

The family information includes the propagator expressions, masses, external scalar products, invariant names, and the numbers of loops, propagators, external momenta and invariant parameters. We encode its symbolic structure with the same fixed 64-component rule and map it to a 32-component family vector $\boldsymbol{h}_F$. A further shared map acts on the index vectors of the chosen masters, and their mean gives the 32-component basis vector $\boldsymbol{h}_M$. This input identifies the basis available for reduction. The target's master coefficients, construction history and recorded seed choices remain outside the input.

Each candidate seed contributes its index and complexity information through a shared map that gives a vector $\boldsymbol{u}_b \in \mathbb{R}^{32}$ for candidate b. To associate a seed with particular target terms, we form

$$\begin{aligned} \omega_{bj} &= \frac{\exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_j/\sqrt{32})}{\sum_{k=1}^{n_T} \exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_k/\sqrt{32})}, \\ \boldsymbol{v}_b &= \sum_{j=1}^{n_T} \omega_{bj} \boldsymbol{h}_j. \end{aligned} \tag{55}$$

The weight ω_{bj} determines how much term j contributes to candidate b's description, and $\boldsymbol{v}_b$ is their weighted mean. This uses the scaled dot-product weighting of Ref. [16]. The weights sum to one over target terms for each seed.

The model then joins the mean and componentwise maximum of the term vectors with the pair, family, basis, seed and weighted-term vectors:

$$\begin{aligned} \boldsymbol{z}_b &= [\text{mean}_j\, \boldsymbol{h}_j,\ \max_j \boldsymbol{h}_j,\ \boldsymbol{h}_{\text{pair}}, \boldsymbol{h}_F, \boldsymbol{h}_M,\ \boldsymbol{u}_b, \boldsymbol{v}_b], \\ g_b &= F_\theta(\boldsymbol{z}_b), \qquad p_b = \frac{1}{1+\exp(-g_b)}. \end{aligned} \tag{56}$$

Here F_θ has a hidden layer with 64 components and one scalar output g_b. The other learned maps each use one layer with 32 components. All hidden layers use the hyperbolic tangent function. The implementation has 28,097 adjustable parameters and shares them across candidates, so reordering the candidate list only reorders the returned scores.

The quantity p_b represents the model's probability that $\boldsymbol{\sigma}_b$ belongs to the recorded sufficient seed set. Several seeds can contribute to the same reduction, so these probabilities are separate and need not sum to one. A high score ranks a seed for selection. The current data establish a valid reference set for each target; they do not establish the probability that a seed belongs to a globally minimal reduction. Exact reduction remains the criterion for deciding whether the selected seeds supply enough equations.

For each candidate we report probabilities for both possible labels:

$$\begin{aligned} P_\theta(y_b = 1 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= p_b, \\ P_\theta(y_b = 0 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= 1 - p_b. \end{aligned} \tag{57}$$

Here y_b is the reference label, F is the family definition and $\mathcal{M}$ is the chosen master basis. Label 1 means membership in the recorded sufficient seed set. Label 0 means absence from that set. The two class probabilities for one seed sum to one, while probabilities for different seeds remain separate.

The binary prediction uses a fixed threshold of one half:

$$\widehat{y}_b = \begin{cases} 1, & p_b \geq \frac{1}{2}, \\ 0, & p_b < \frac{1}{2}. \end{cases} \qquad q_b = \widehat{y}_b p_b + (1 - \widehat{y}_b)(1 - p_b). \tag{58}$$

The value $\widehat{y}_b$ is the returned label, and q_b is the model's estimated probability that this label agrees with the reference label. For example, $p_b = 0.90$ gives label 1 with confidence $q_b = 0.90$, while $p_b = 0.08$ gives label 0 with confidence $q_b = 0.92$. These values illustrate the output rule; they are not trained results. Candidate selection uses p_b, while q_b describes confidence in the returned binary decision.

The predicted seed set is

$$\widehat{\mathcal{S}}(\mathcal{T}) = \{\boldsymbol{\sigma}_b \in \mathcal{C} : \widehat{y}_b = 1\}. \tag{59}$$

Its size can vary between targets. Exact reduction must establish whether the identities from this set suffice to reduce the target. The probability q_b concerns one reference-membership decision; it is not a probability that the complete selected set gives a valid or minimum-cost reduction. Section 8 compares these confidence values with their observed agreement with the reference labels.

7.2 Training labels and the 70:30 data division

For each target, let $\mathcal{S}_\mathcal{T} \subseteq \mathcal{C}$ be its recorded sufficient seed set. We define one label per candidate:

$$y_b(\mathcal{T}) = \begin{cases} 1, & \boldsymbol{\sigma}_b \in \mathcal{S}_\mathcal{T}, \\ 0, & \boldsymbol{\sigma}_b \notin \mathcal{S}_\mathcal{T}. \end{cases} \tag{60}$$

A zero label records absence from this particular set. Another valid reduction can still use that seed. Targets with empty reference seed sets retain zero labels for all candidates, and their checks are kept separate when a comparison requires at least one positive label.

We assign each complete target, its candidate list and all its labels to one side of the 70:30 division. We also keep each complete family together and group four-propagator families with the same kinematic zero and equality pattern across all mass assignments. This includes matching patterns among the original massless boxes. These conditions prevent related targets from the same full family profile from being used on both sides.

A reproducible assignment gives 168,330 training targets in 693 families and 72,142 test targets in 307 families. The training fraction is 69.99983%. The ratio refers to complete target examples; each example carries its whole seed-label vector, whose length depends on the family. This model-specific division uses the complete collection and leaves the original data divisions in Section 6 available as provenance. Shared lower-sector integrals can still occur across the two sets. We assigned two complete two-loop families to training and one to testing with the fixed split seed. These three families alone cannot establish performance at high loop orders.

The training objective is the average binary cross-entropy,

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{\mathcal{T} \in \mathcal{D}_{\text{train}}} \frac{1}{n_C(\mathcal{T})} \sum_{b=1}^{n_C(\mathcal{T})} \left[y_b \log p_b + (1 - y_b) \log(1 - p_b) \right]. \tag{61}$$

Here $\mathcal{D}_{\text{train}}$ is the training target set and $n_C(\mathcal{T})$ is the candidate count for that target. Each target has equal weight in this objective. We retain all candidate labels and use no additional positive-class weighting. We trained the model with AdamW [17], with an initial learning rate of 10^{-3}, weight decay of 10^{-4} and ten passes through the training data. We fixed these settings before using the test set. Training completed all ten passes and made 210,420 parameter updates from 1,683,300 target presentations. The final saved parameters define the model used throughout the test.

For example, suppose five candidate seeds accompany a target and its recorded sufficient set contains the first and third seeds. The label vector is then

$$\boldsymbol{y}(\mathcal{T}) = (1, 0, 1, 0, 0). \tag{62}$$

The model receives the target, public family information, chosen basis and all five candidate seeds. Their labels enter the comparison with the predictions after the forward calculation; the labels and the target's master expansion remain outside the model input.

Starting from randomly initialized parameters, the model produces a probability for every candidate in a training example. A reference label of 1 contributes $-\log p_b$ to the loss, while a label of 0 contributes $-\log(1 - p_b)$. The loss uses these continuous probabilities during parameter updates. The threshold in Eq. (58) applies when reporting decisions.

We differentiate the loss with respect to the model parameters and average the gradients from eight targets before each update. A gradient describes how a change in a parameter changes the loss. The training procedure limits the combined gradient norm to one and uses AdamW to update the 28,097 parameters. The last group in a pass uses its actual number of targets when fewer than eight remain. The training examples change order between passes. Ten passes use each training target ten times, and the saved parameters after the final pass define the model for the test comparison. The held-out targets do not enter parameter updates or threshold selection.

7.3 Completed training and implementation checks

Training began from randomly initialized parameters and completed the fixed ten-pass schedule. The mean loss recorded during the first pass was 0.0111521, and the mean during the last pass was 0.00148954. Figure 1 shows the recorded loss for each pass. These values average the losses encountered as the parameters changed within that pass. They describe the training history and do not measure performance on unseen targets.

Before training, the preparation checked every target-label correspondence and verified that every recorded seed appears in its candidate pool. Implementation checks verified the binary labels, both class probabilities, the confidence values and the parameter derivatives. They also confirmed that the input retains coefficient and term-order information and that a reordering of candidates only reorders their scores. During training, the held-out targets supplied no parameter updates, threshold choices or stopping decisions.

The implementation reads examples in bounded groups and scores candidate seeds and ordered term pairs in blocks. It retains every ordered pair in Eq. (54), so the pair count grows as $n_T(n_T - 1)$. The current bounds are 512 terms, 10,000 candidates and 16 indices per integral. Inputs beyond these bounds are rejected. The trained model now permits a direct comparison of its predicted seed sets with the recorded sets in the separate test collection.

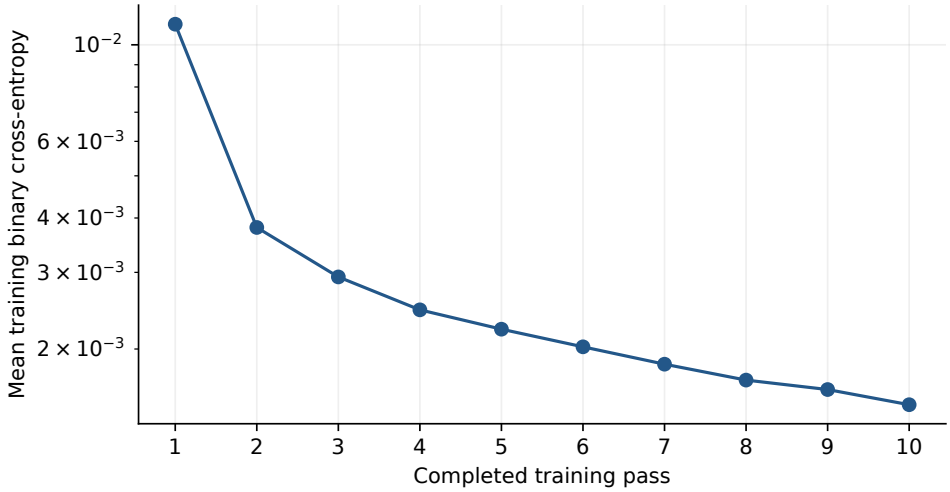


Figure 1: Mean training binary cross-entropy during each of the ten passes. Each training target contributes equal weight to the mean. The vertical axis uses a logarithmic scale. The curve records losses during parameter updates; it is not a validation curve.

8 Seed-Selection Test Results

To measure how accurately the trained model identifies the recorded seed choices, we evaluated the saved model on all 72,142 held-out targets from 307 families. We kept its parameters and the threshold $p_b = 1/2$ fixed throughout the test. The test contained 16,124 individual integrals and 56,018 integral expressions. Its 306 one-loop families and one two-loop family supplied 33,828,275 candidate-label comparisons. Of these labels, 201,885 were positive, corresponding to 0.597% of the comparisons. The model received the same public inputs defined in Section 7; the recorded seed sets entered only the subsequent comparison. The accompanying test records contain every predicted probability, the reference labels and the figures used here [18].

8.1 What the test measures

The reference set $\mathcal{S}_\mathcal{T}$ supplies one known sufficient choice of seeds for target $\mathcal{T}$. The model returns $\widehat{\mathcal{S}}(\mathcal{T})$ from Eq. (59). We count a selected reference seed as a true positive, a selected seed absent from the reference as a false positive, and a missed reference seed as a false negative. A true negative is an unselected candidate with reference label zero. The corresponding counts are $N_{\mathrm{TP}}, N_{\mathrm{FP}}, N_{\mathrm{FN}}$ and N_{TN}. These names describe agreement with the recorded choice. A seed absent from that choice can still occur in another valid reduction.

Summing these counts across all targets gives the pooled measures

$$\begin{aligned} A &= \frac{N_{\mathrm{TP}} + N_{\mathrm{TN}}}{N_{\mathrm{TP}} + N_{\mathrm{TN}} + N_{\mathrm{FP}} + N_{\mathrm{FN}}}, \\ P &= \frac{N_{\mathrm{TP}}}{N_{\mathrm{TP}} + N_{\mathrm{FP}}}, \qquad R = \frac{N_{\mathrm{TP}}}{N_{\mathrm{TP}} + N_{\mathrm{FN}}}, \\ F_1 &= \frac{2N_{\mathrm{TP}}}{2N_{\mathrm{TP}} + N_{\mathrm{FP}} + N_{\mathrm{FN}}}. \end{aligned} \tag{63}$$

Here A is candidate-label accuracy, P is the fraction of selected candidates that belong to the reference, and R is the fraction of reference seeds selected. The quantity F_1 combines precision and recall. The prevalence of zero labels makes accuracy sensitive to correctly rejecting large numbers of candidates, so we also examine precision and recall, following the evaluation approach

discussed in Ref. [19]. Assigning zero to every candidate would already give 99.403% accuracy and zero recall.

We also require a measure of the predicted set for each target. Its exact agreement fraction is

$$E = \frac{1}{N_{\text{test}}} \sum_{\mathcal{T} \in \mathcal{D}_{\text{test}}} \mathbf{1}\left[\widehat{\mathcal{S}}(\mathcal{T}) = \mathcal{S}_{\mathcal{T}}\right], \tag{64}$$

where $\mathcal{D}_{\text{test}}$ contains the $N_{\text{test}} = 72{,}142$ test targets and $\mathbf{1}$ is one when its stated condition holds and zero otherwise. Reference containment instead asks whether the predicted set contains every reference seed while allowing additional candidates. We calculate recall and containment for the 72,140 targets with nonempty reference sets. Both targets with empty reference sets remain in accuracy and exact-set agreement. Whenever a precision denominator vanishes, its target-level value is undefined and is excluded from that particular average.

8.2 Binary seed decisions

At the fixed threshold, the model correctly selected 189,999 reference seeds and missed 11,886. It selected 8,466 candidates absent from the corresponding reference sets and correctly rejected 33,617,924 zero-labelled candidates. These counts give 99.940% pooled accuracy, 95.734% precision, 94.112% recall and 94.916% F_1. Figure 2 displays the counts and their fractions within each reference class. The high zero-label accuracy must be read together with the positive-seed recall.

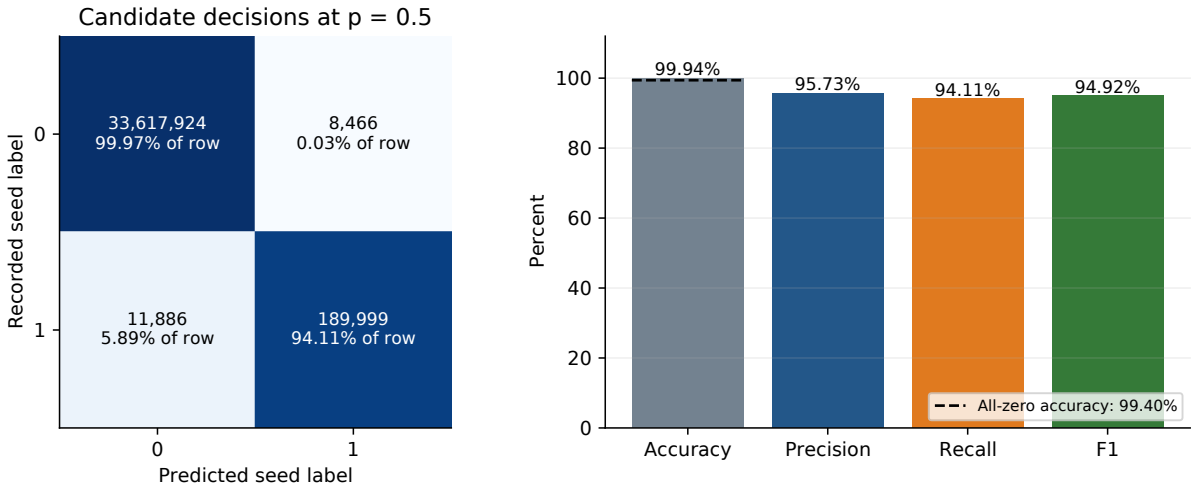


Figure 2: Held-out candidate decisions at the fixed probability threshold of one half. The matrix gives counts and percentages normalized within each reference-label row. The right panel gives pooled classification measures. The dashed mark applies only to accuracy and shows the result of assigning zero to every candidate.

The predicted set exactly matched the reference for 81.916% of all test targets. It contained every reference seed for 86.762% of targets with nonempty reference sets. These two measures distinguish an identical predicted set from a larger set that still contains the recorded choice. The fixed model's mean test binary cross-entropy was 0.003530, with equal weight for each target as in Eq. (61).

8.3 Recovering seeds from the ranked candidates

The returned probabilities also order the available seeds, which allows a solver to request further candidates after an initial selection. For the k highest-ranked candidates, write the selected set as $\widehat{\mathcal{S}}_k(\mathcal{T})$ and define

$$R_k(\mathcal{T}) = \frac{\left|\widehat{\mathcal{S}}_k(\mathcal{T}) \cap \mathcal{S}_{\mathcal{T}}\right|}{|\mathcal{S}_{\mathcal{T}}|}, \qquad |\mathcal{S}_{\mathcal{T}}| > 0. \tag{65}$$

The list contains at most k seeds when a family has fewer candidates. Equal probabilities use the lexicographic order of seed index vectors, which gives a fixed ordering without reference-label information.

For $k = 1, 5, 10, 20$, the target-averaged recall was 48.246%, 97.669%, 99.795% and 99.916%, respectively. The corresponding fractions of targets whose entire reference sets appeared in those lists were 21.462%, 90.654%, 99.461% and 99.831%. Figure 3 also separates individual integrals from integral expressions, since they can require different numbers of reference seeds.

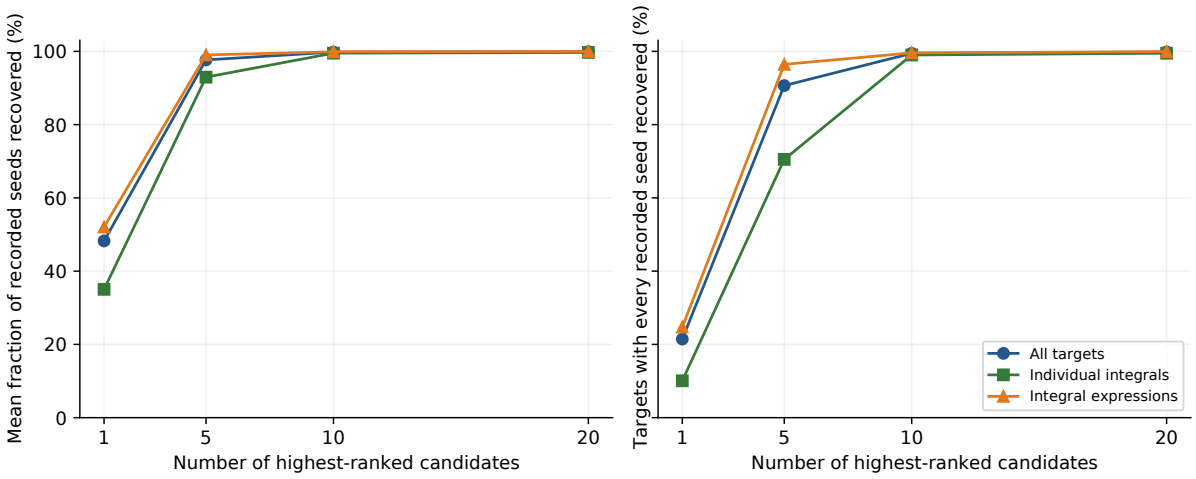


Figure 3: Recovery from the highest-ranked candidates. The left panel averages the fraction of reference seeds recovered per target. The right panel gives the fraction of targets whose complete reference sets appear in the candidate list. The averages exclude the two empty-reference targets. Lines join the four measured candidate counts.

To summarize each target's full ranking, we calculate average precision over distinct predicted probabilities,

$$\mathrm{AP}(\mathcal{T}) = \sum_j (R_j - R_{j-1}) P_j, \qquad R_0 = 0. \tag{66}$$

Here P_j and R_j are precision and recall after including every candidate at the jth distinct probability, in descending order. This definition groups ties and weights precision by the additional recall at each threshold [20]. The mean average precision over nonempty targets was 0.988090, and its mean over equally weighted family averages was 0.985413. Figure 4 gives the pooled precision–recall curve. It serves as a diagnostic of the ranking; the reported binary decisions retain the original threshold.

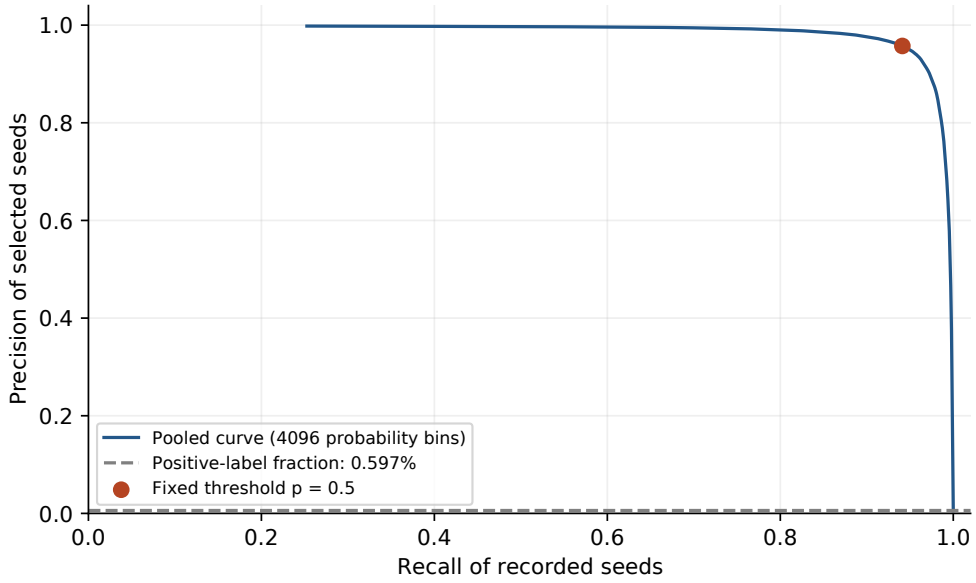


Figure 4: Pooled precision and recall across candidate-label comparisons. The curve uses 4,096 probability bins to keep memory use bounded, so it approximates the continuously varying threshold curve. The marked point uses the exact counts at $p_b = 1/2$. The dashed line gives the positive-label fraction. The target average precision reported in the text uses exact distinct probabilities, without this binning.

8.4 Confidence and variation between integral families

The confidence q_b in Eq. (58) estimates agreement of the returned label with the reference. We group decisions into intervals of width 0.1 in q_b, separately for returned labels zero and one, and compare mean confidence with the observed fraction of agreeing labels. For returned label one, the mean confidence was 95.523% and the observed agreement was 95.734%. For returned label zero, these values were 99.973% and 99.965%. Figure 5 displays the populated intervals. A point on its diagonal has matching mean confidence and observed agreement. The target-averaged squared probability error was 0.000761, where the error for a target is $n_C^{-1} \sum_b (p_b - y_b)^2$.

The pooled precision and recall were 96.014% and 95.801% for individual-integral targets, and 95.606% and 93.351% for integral expressions. The one-loop group gave 96.426% precision and 94.227% recall. The 144 targets in the massless two-loop propagator family gave 11.290% precision and 41.553% recall. Its predicted set exactly matched the reference for 0.000% of targets. The highest-ranked 20 candidates contained the full reference set for 48.951% of its nonempty-reference targets. Figure 6 shows the spread of family-averaged ranking quality and the pooled seed measures at each loop order. Since the two-loop result comes from one family, it provides a limited test of transfer beyond the dominant one-loop collection.

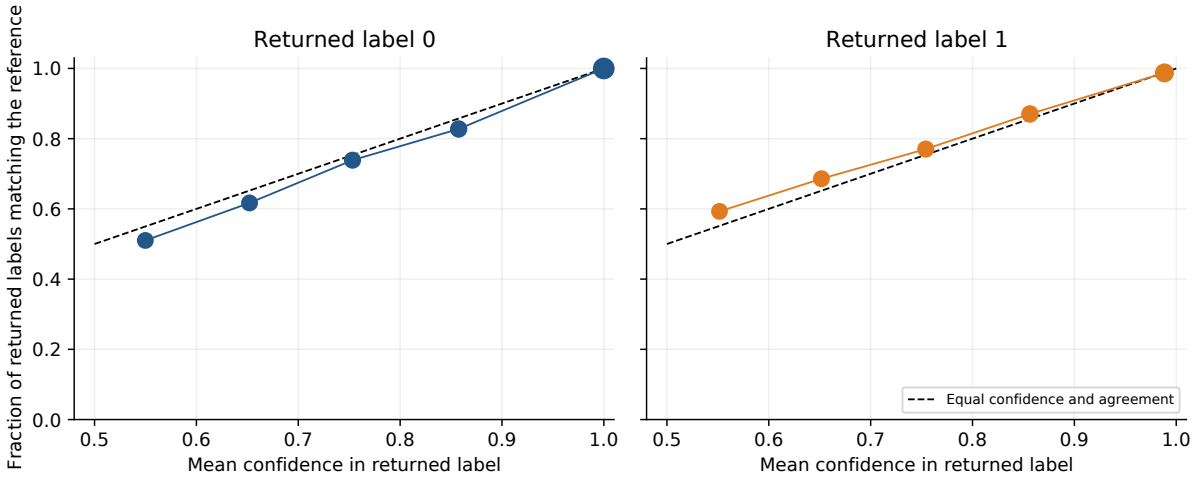


Figure 5: Confidence in each returned binary label and observed agreement with the recorded label. Each point represents one populated confidence interval, and larger points represent more candidate decisions. The dashed diagonal indicates equal confidence and agreement. Separate panels retain the distinction between selected seeds and rejected candidates.

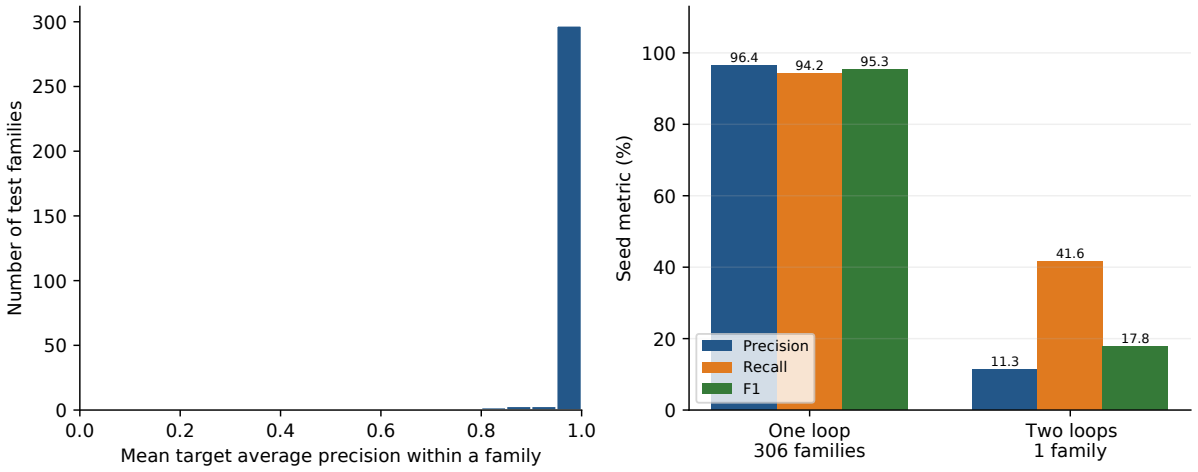


Figure 6: Variation across the 307 held-out families. The left panel gives the distribution of mean target average precision within each family. The right panel pools candidate decisions separately for 306 one-loop families and one two-loop family at the fixed threshold. The single two-loop family cannot establish general high-loop performance.

8.5 Implications for IBP reduction

The test confirms the stated level of agreement with the constructed reference reductions. The model parameters remained unchanged during all test calculations. Encoding the target terms and computing their seed probabilities took a mean of 4.300 milliseconds per target on one CPU thread; this timing excludes file reading, metric calculations and IBP algebra.

The test families are separate from the training families under the grouping rule in Section 7.2. Most of the full collection shares a one-loop box topology, and lower-sector integrals can occur in both divisions. The aggregate measures mainly reflect those one-loop examples. The lower precision and recall of the held-out two-loop family show that the present model has limited transfer to that family. The measurements apply to this constructed collection and its bounded candidate pools. They do not determine a globally minimal seed set. A subsequent reduction study must pass the selected seeds to Kira, check whether their identities reduce each target in the chosen master basis, and include any extra seeds needed to complete the reduction. That study will determine the effect on equation counts, memory and total reduction time.

9 Machine Learning Model for Reduction Propagation

The seed model selects identities from which a reduction can proceed. Once these equations are available, the reduction model selects the next row operation from a supplied list of allowed steps. We use the current coefficient matrix and its integral labels to describe each choice, and the model returns a probability for every listed step. The step with the largest probability gives the proposed next operation. Exact arithmetic then updates the matrix, which supplies the input for the next choice. This section describes the model and its training on the reductions constructed in Section 6.

9.1 Current equations and allowed reduction steps

Let $A^{(n)}$ be the coefficient matrix after n reduction steps. A column j corresponds to an integral $I_{\boldsymbol{a}^{(j)}}$, and a row contains its coefficients in an identity or in the target expression currently being reduced. The chosen master basis specifies the columns protected from elimination. An action $a = (q, r, k)$ uses source row q to eliminate column k from destination row r:

$$A_{rj}^{(n+1)} = A_{rj}^{(n)} - \frac{A_{rk}^{(n)}}{A_{qk}^{(n)}} A_{qj}^{(n)}, \qquad q \neq r, \quad A_{qk}^{(n)} \neq 0, \quad A_{rk}^{(n)} \neq 0. \tag{67}$$

All other rows remain unchanged during this elementary step. If both rows are identities, this is an elementary row operation. If row r represents a target, row q must be a zero identity so that the target retains its value. The solver supplies the allowed action set $\mathcal{A}_n$, enforcing its integral ordering, master basis and row roles. The model ranks this finite list; completeness of the list remains the responsibility of the solver.

The stored examples describe a particular instance of this matrix. They contain one changing target row and the source identities available for eliminating its largest remaining nonmaster integral under Eq. (36). Each candidate specifies one of these identities, the target row and that integral. After a choice, the source identities remain fixed and the target row changes. The prediction interface also accepts several changing rows, while the present training data cover the stored target-row procedure. Performance on Kira's full evolving system will require a separate study with its admissible actions and ordering [9].

9.2 Information supplied to the model

For each allowed action, we construct a vector $\boldsymbol{f}_a$ from the current matrix, the pivot integral and the family definition. The pivot description includes its indices, sector membership, and the values of $r, s, t, n_{\mathrm{dot}}$ in Eq. (14). Descriptions of the source and destination rows give their nonzero counts, master content and the mean and largest integral complexities. A matrix summary gives its number of rows, columns and nonzero entries. The public family definition supplies the numbers of loops and kinematic parameters together with the propagators and scalar-product relations. We encode their written form through the same fixed signed token counts described in Section 7.1, using 32 components here. This encoding admits collisions between different groups of symbols. We transform nonnegative counts with $\log(1+v)$, where v is the count, and each signed index a_i with $\operatorname{sgn}(a_i)\log(1+|a_i|)$. The representation allows up to 16 indices and records which positions belong to the family. These definitions give 132 features for each candidate.

The current records supply coefficients in $\mathbb{F}_P$, the field of integers modulo a prime P. We use these coefficients to calculate the overlap of each source row with its destination, the number of terms that the proposed operation would cancel or introduce, and its immediate arithmetic count. For a source with n_q nonzero entries, this count is $1 + 2n_q$: one division and n_q multiplications and subtractions, using the convention in Section 6. These quantities require one sparse trial row update per action. They use only the current matrix and do not inspect later reduction steps or reference master coefficients. A symbolic coefficient can vanish at the chosen sample, so the model uses counts recomputed from the sampled rows. The data audit retains the distinction from the original symbolic support counts. Structural counts and integral descriptions carry this information into the model. The numerical size of a modular residue does not represent the magnitude of a real coefficient.

The representation compresses the supplied matrix into these features, so different states can have the same description. It includes local cancellation information and a summary of the candidate set. These quantities describe the current choices without enumerating complete reduction paths. Forming these features and evaluating the model contribute to the eventual runtime and must be included when measuring a reduction speedup.

9.3 Action probabilities and confidence

A shared map ϕ_θ converts each candidate vector into a 32-component vector,

$$\boldsymbol{h}_a = \tanh(W_1 \boldsymbol{f}_a + \boldsymbol{b}_1), \qquad \boldsymbol{u} = \frac{1}{|\mathcal{A}_n|} \sum_{a \in \mathcal{A}_n} \boldsymbol{h}_a, \qquad \boldsymbol{v} = \max_{a \in \mathcal{A}_n} \boldsymbol{h}_a. \tag{68}$$

Here θ denotes the adjustable parameters, W_1 is a learned matrix, $\boldsymbol{b}_1$ is a learned offset, and the maximum acts on each component separately. Sharing the map and combining candidates through averages and maxima makes the scores independent of their position in the supplied list [15]. The model joins $\boldsymbol{h}_a, \boldsymbol{u}, \boldsymbol{v}$ and applies a second map with 64 hidden components to obtain a scalar score $g_\theta(a)$. For a fixed family, master basis and finite-field sample, it returns

$$\begin{aligned} p_\theta(a \mid A^{(n)}, \mathcal{A}_n) &= \frac{\exp g_\theta(a)}{\sum_{b \in \mathcal{A}_n} \exp g_\theta(b)}, \\ \widehat{a}_n = \arg\max_{a \in \mathcal{A}_n} p_\theta(a), &\qquad q_n = p_\theta(\widehat{a}_n). \end{aligned} \tag{69}$$

Thus the probabilities sum to one over the supplied alternatives, and q_n is the reported confidence for the selected step. With one allowed step the probability is one by construction, which provides

no evidence of learned discrimination. An empty candidate list returns no proposed operation; the solver must determine whether the reduction is complete or further identities are required.

Training gives these probabilities an interpretation as agreement with the recorded next choice, conditional on the supplied state and actions. Several choices can give valid reductions, and an alternative choice may even lead to a shorter sequence. The reference paths establish valid reductions and their recorded costs; they do not establish global minima. The reported confidence therefore requires a later calibration check against held-out choices and does not certify exactness, minimal cost or the success of a complete reduction. Equation (67) and the solver's checks retain responsibility for the algebra.

9.4 Training examples and separation from testing

We retain the same grouped family division used for the seed model: 693 families with 168,330 targets supply training records, while 307 families with 72,142 targets remain reserved for testing. Every state from one target stays with that target's family. We read the row before each recorded step and its complete stored list of admissible source equations. The chosen equation supplies the reference action $a_n^\star$. The row after the step and the stored operation counts check the data preparation but remain outside the predictive inputs. The target's construction history, final master coefficients and future choices also remain outside those inputs.

For the set $\mathcal{D}_{\mathrm{choice}}$ of training states with at least two allowed actions, the objective is

$$\mathcal{L}_{\mathrm{red}}(\theta) = -\frac{1}{|\mathcal{D}_{\mathrm{choice}}|} \sum_{(A,\mathcal{A},a^\star)\in\mathcal{D}_{\mathrm{choice}}} \log p_\theta(a^\star \mid A, \mathcal{A}). \tag{70}$$

Each such state has equal weight. States with one admissible action have zero loss and remain in the data audit. The recorded reference choice is the training label; absence of a label on another action does not establish that its reduction is invalid.

We use AdamW [17] with a learning rate of 10^{-3}, weight decay 10^{-4}, batches of 256 states and a gradient norm limit of one. The fixed schedule uses ten passes through the training choice states. The data preparation replays the selected row operations with exact modular arithmetic and verifies the final recorded master rows. Training uses shuffled states from bounded groups of families and retains resumable model and optimizer parameters. The held-out families do not enter parameter updates, stopping decisions or confidence adjustment.

9.5 Completed training and present scope

Training used 585,502 recorded states from the 168,330 targets in 693 training families. Of these states, 455,341 supplied at least two allowed actions and 130,161 supplied one action. The prepared collection contains 2,085,062 candidate appearances, with at most 16 actions in one state. The model uses 132 features per candidate and has 10,529 adjustable parameters.

All selected reference operations reproduced their saved sampled next rows, and the replay checks verified trace continuity and the final master support. The audit recorded 108 states in which the stored symbolic support counts differed from counts in the sampled matrix. Coefficients can vanish at the chosen kinematic point, so these states use their exact sampled counts as described above.

The fixed schedule completed ten passes with 4,553,410 presentations of choice states and 18,006 parameter updates. The mean cross-entropy recorded during parameter updates changed from 0.186664 in the first pass to 0.115099 in the tenth pass. The corresponding fraction of choices

matching the recorded action changed from 97.983% to 99.739%. These values describe the training history. Section 10 evaluates the final parameters on the reserved families.

Implementation checks verified exact row cancellation, unchanged predictions when the candidate list or matrix rows are reordered, invariance of the structural features under nonzero row rescaling, and the exclusion of future labels from the inputs. The saved model also reproduced the feature encoding of a recorded training state and returned probabilities summing to one. A selected operation passed an independent exact row-update check. The accompanying implementation retains the trained parameters and the data provenance [21].

These records cover one changing target row with fixed source identities, predominantly in one-loop families. The model can now rank the supplied steps at a current state. Section 10 tests complete learned target-row sequences on reserved families and compares confidence with recorded-action agreement. The training measurements above describe fitting on the training data. Minimum operation counts and performance in complete Kira jobs require separate studies.

10 Tests of the Reduction-Propagation Model

To test the model in Section 9, we used its saved parameters on the 72,142 targets in the 307 reserved families. We tested the next choice at a recorded matrix and the complete sequence obtained when the model receives the matrix produced by its own preceding choice. We used the saved parameters and raw output probabilities throughout this evaluation. The accompanying records retain the predictions, chosen identities and resulting coefficients [22].

10.1 Test inputs and next-step agreement

The test contains 16,124 individual integrals and 56,018 integral expressions. There are 306 one-loop families and one two-loop family, the massless propagator family in Eq. (24). Their recorded reductions contain 247,422 states, of which 194,347 have at least two allowed steps and 53,075 have one forced step. Every target stays in the family division defined in Section 7.2.

For each recorded state, the model receives the row before the step and all its admissible source equations. We compare the selected action $\widehat{a}_i$ with the recorded action $a_i^\star$, where i labels the state. Let $\mathcal{Q}$ be the set of states with at least two actions and $N_{\mathcal{Q}} = |\mathcal{Q}|$. Their next-step agreement is

$$\mathrm{Acc}_1 = \frac{1}{N_{\mathcal{Q}}} \sum_{i \in \mathcal{Q}} \mathbf{1}[\widehat{a}_i = a_i^\star]. \tag{71}$$

The indicator $\mathbf{1}$ equals one when its stated condition holds and zero otherwise. Forced steps have probability one without a learned choice, so the primary metrics exclude them. We also check equality of the sampled next rows, since different equations can give the same update.

The model matched 95.469% of recorded choices, placed the recorded choice among its three highest scores in 99.990% of states, and produced the same next sampled row in 95.476% of states. Mean agreement per target with a choice was 96.189%, and the mean across families with choices was 96.019%. Figure 7 compares these state-level results with a uniform random choice and the fixed local rule defined below. The random-choice value averages $1/|\mathcal{A}_i|$ over the same states, where $\mathcal{A}_i$ is the supplied action set.

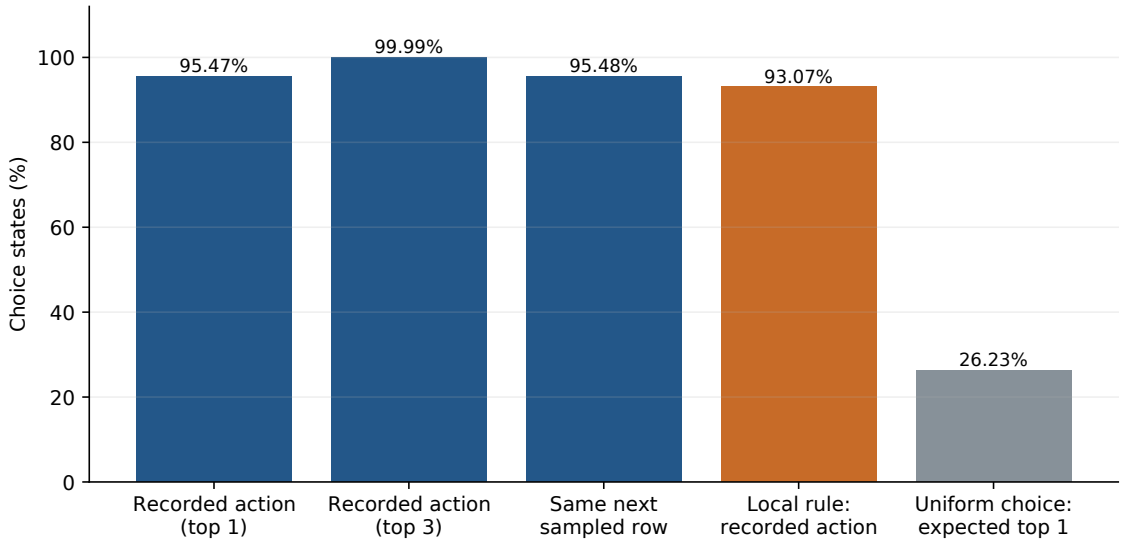


Figure 7: Next-step comparisons on states with at least two actions. The first two bars concern the recorded equation; the third concerns equality of the resulting sampled row. Other valid actions can give different intermediate rows.

10.2 Confidence and variation between families

Let $p_i(a)$ be the probability assigned to action a in state i. Its largest value $p_i^{\max}$ is the selected-action confidence from Eq. (69), and $c_i = \mathbf{1}[\widehat{a}_i = a_i^\star]$ records agreement with the reference action. We compare their means in ten equal intervals of $[0, 1]$, including confidence one in the last interval. The empirical calibration error is

$$E_{\mathrm{cal}} = \sum_{\beta : N_\beta > 0} \frac{N_\beta}{N_{\mathcal{Q}}} |\overline{p}_\beta^{\max} - \overline{c}_\beta|, \tag{72}$$

where N_β is the number of choice states in interval β and the overbars denote their means. This follows the reliability-bin approach of Ref. [23] without fitting a calibration map.

We also compare the full probability distribution with the recorded choice. The mean negative log probability and multiclass Brier score are

$$\begin{aligned} L_{\mathrm{test}} &= -\frac{1}{N_{\mathcal{Q}}} \sum_{i \in \mathcal{Q}} \log\Big[\max\{p_i(a_i^\star), 10^{-38}\}\Big] , \\ B_{\mathrm{test}} &= \frac{1}{N_{\mathcal{Q}}} \sum_{i \in \mathcal{Q}} \sum_{a \in \mathcal{A}_i} \left(p_i(a) - \mathbf{1}[a = a_i^\star]\right)^2 . \end{aligned} \tag{73}$$

The probability floor defines the logarithm if a returned probability rounds to zero. Mean confidence was 91.760%, compared with 95.469% agreement, while $E_{\mathrm{cal}} = 0.0371$, $L_{\mathrm{test}} = 0.1180$ and $B_{\mathrm{test}} = 0.0815$. Figure 8 shows agreement and state counts within each interval. These probabilities concern the recorded next choice and do not give a probability that the whole reduction succeeds.

Exact ties in the largest returned probability occurred in 5.656% of choice states. The implemented rule resolves them by equation identifier. Choosing uniformly among tied maxima would give 92.681% next-step agreement. We retain the implemented rule in all predictions and paths; the additional count separates score discrimination from tie resolution.

One-loop choice agreement was 95.482%. The two-loop family gave 90.213% across 470 choice states and reproduced the recorded next sampled row in 92.553% of those states. Figure 9

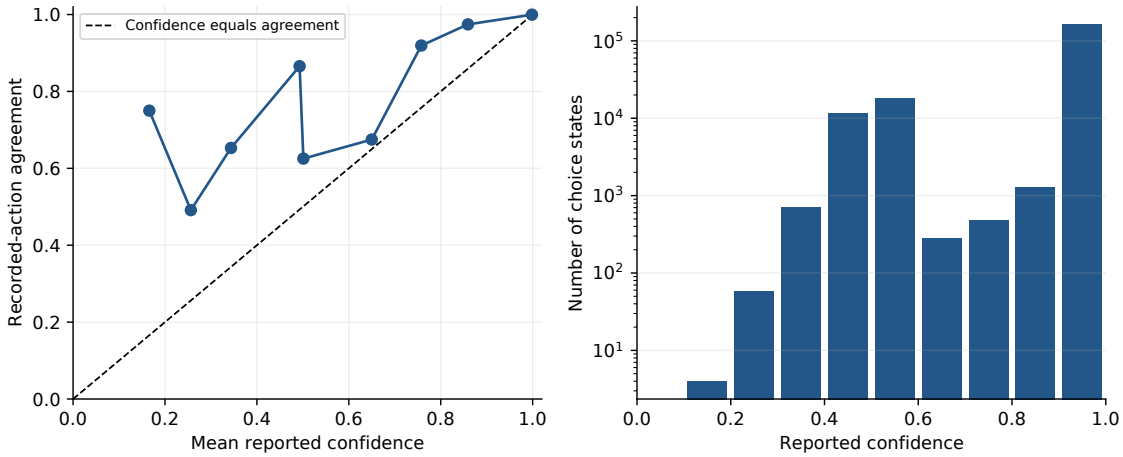


Figure 8: Confidence compared with recorded-action agreement. Forced steps are excluded. The diagonal denotes equal confidence and agreement, and the right panel gives each bin's population.

displays the family distribution and loop-order comparison. The training set also contains the equal-mass two-loop sunrise and vacuum families. The two-loop test therefore measures transfer to the reserved massless propagator topology within the available two-loop collection.

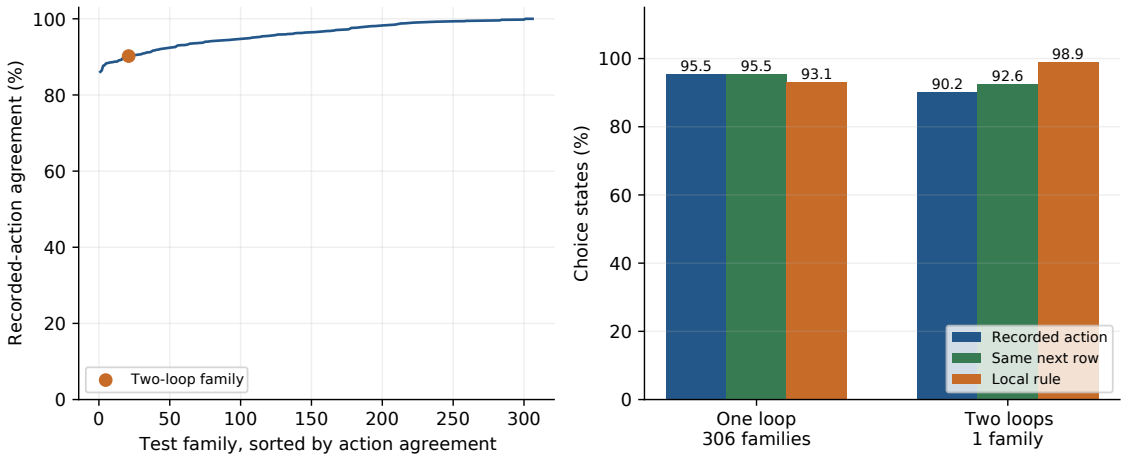


Figure 9: Recorded-action agreement across families and comparison of one-loop and two-loop states with choices. The marked point identifies the massless two-loop propagator family.

10.3 Complete reduction sequences and coefficient checks

Agreement at a recorded state measures a single decision. To check how successive decisions affect the reduction, we also start from each target and let the model select every step. After each exact update, it receives the row that its own choice produced. The next pivot is the largest remaining nonmaster integral under Eq. (36). We obtain the candidate equations from the family's stored identities with that leading integral and a nonzero sampled pivot. Rebuilding this set reproduces every candidate list along the recorded reference paths.

For comparison, a fixed local rule tries the same candidate equations and selects the result with the fewest nonmaster terms. It resolves ties first by the total number of nonzero terms, then by the arithmetic count $1 + 2n_q$ for a source row with n_q nonzero coefficients, and finally by

equation identifier. Both policies receive the same target, master basis, source equations and integral order. Neither policy receives the final master coefficients or later reference steps.

A sequence ends when the row contains only masters. We also stop if no supplied equation can cancel the next integral, if the row exceeds 4,096 nonzero terms, or if the sequence reaches 4,096 updates. The model's input bounds in Section 9 also apply. These bounds prevent an unsuccessful sequence from growing without limit. We retain every unsuccessful target in the results and supply no reference-path fallback.

Each selected update uses exact finite-field arithmetic and passes a cross-multiplied row check. We then repeat its sequence of source identities at all three stored points and primes, evaluating the original symbolic coefficients at each point. A vanishing source pivot marks an unverified sequence at that point. Successful verification requires a completed reduction and equality with every stored master coefficient at all three samples. These checks use the finite-field approach discussed in Section 3 [11]. Agreement at the sampled points supplies evidence for these numerical reductions; a symbolic result at generic parameters would require additional reconstruction or symbolic verification.

The learned policy passed all three coefficient comparisons for 72,026 of 72,142 targets (99.839%), while the local rule passed for 71,463 targets (99.059%). The remaining 116 learned paths stopped at a row with no allowed next action. Every completed learned path matched all three samples, and all attempts stayed within the prescribed size and step bounds. The local rule stopped with no allowed action for 664 targets. It matched the first sample for 71,478 targets, of which 15 failed the additional coefficient comparisons. These cases show why a completed reduction at one sample requires checks at other points.

An allowed identity can introduce a smaller nonmaster integral for which the stored pool has no usable source equation. The row update remains valid, while the supplied equations cannot continue that selected sequence. In the two-loop family, the learned policy passed 142 of 144 targets and the local rule passed 144. Figure 10 separates the loop orders and compares the first-sample results with the three-sample results.

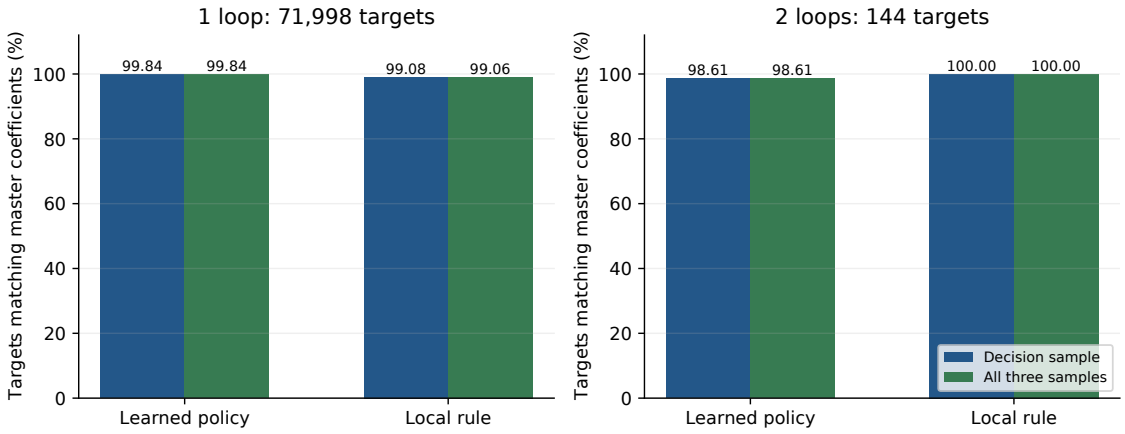


Figure 10: Complete reductions that reproduce the stored master coefficients. Each percentage uses all targets in its loop-order group, including unsuccessful paths.

10.4 Operation counts and measured reduction time

To measure the work along a completed path, we count one division and one multiplication and subtraction per nonzero source coefficient, as in Section 6. If K is the number of selected steps and n_{q_i} is the number of nonzero coefficients in the source selected at step i, the committed

operation count is

$$C_{\text{path}} = \sum_{i=1}^{K} (1 + 2n_{q_i}). \tag{74}$$

Both policies also make trial updates to compare candidates. Their corresponding count is

$$C_{\text{trial}} = \sum_{i=1}^{K} \sum_{a \in \mathcal{A}_i} (1 + 2n_{q(a)}), \tag{75}$$

where $q(a)$ is the source equation of candidate a and $\mathcal{A}_i$ is the action set at step i. We report C_{path} and $C_{\text{path}} + C_{\text{trial}}$ separately. These counts cover row arithmetic and exclude neural-network calculations, row checks, equation generation, symbolic simplification and reconstruction.

The recorded construction gives a valid reference count C_{ref} under the same arithmetic convention. Its minimality has not been established, so a ratio $C_{\text{path}}/C_{\text{ref}}$ measures cost relative to this recorded path. Among successful learned paths with $C_{\text{ref}} > 0$, 155 used fewer committed operations, 71,553 used the same number, and 318 used more. For the 71,448 targets that both policies completed and that have $C_{\text{ref}} > 0$, the learned policy used fewer operations than the local rule in 217 cases, the same number in 64,701 cases, and more in 6,530 cases.

Figure 11 uses this common set so that both curves describe the same targets. It shows how the trial updates add work even when the selected path has the same operation count as the reference. The separate success comparison retains the targets excluded from these completed-path ratios.

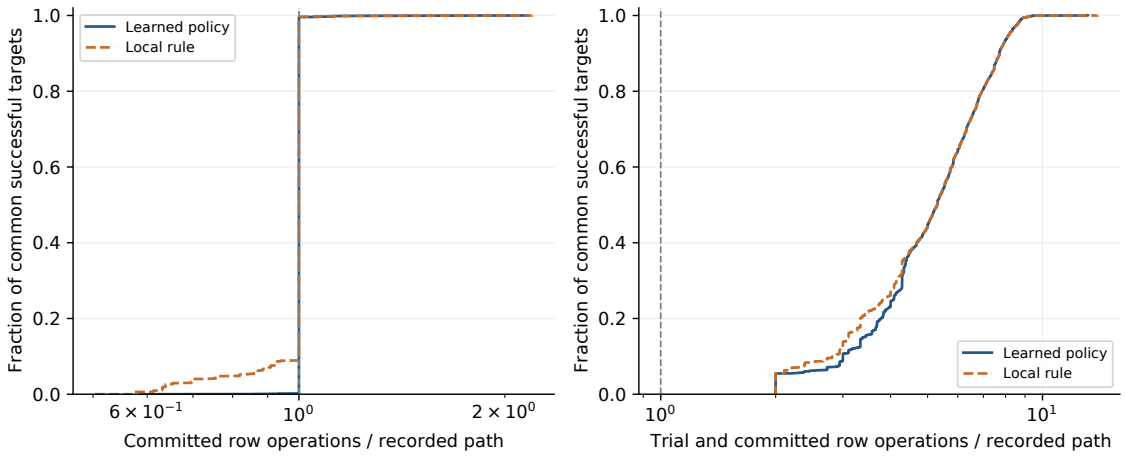


Figure 11: Cumulative fractions of common successful targets below each operation-count ratio. The reference is the recorded valid path. The left panel includes selected row updates; the right panel also includes candidate trial updates.

We measured each complete attempted sequence on one CPU, including state construction, candidate selection, exact updates and the per-step row checks. The measurement excludes file access, building the shared source-equation pool and the subsequent replays at the stored samples. We alternated the order in which the two policies ran from one target to the next. The mean times over all targets, including unsuccessful attempts, were 8.4426 ms for the learned policy and 0.4262 ms for the local rule, giving a ratio of 19.81. The corresponding medians were 7.2665 ms and 0.3488 ms. Figure 12 shows the distribution and its summary values. These measurements include the current model's feature construction and scoring cost and describe this target-row implementation.

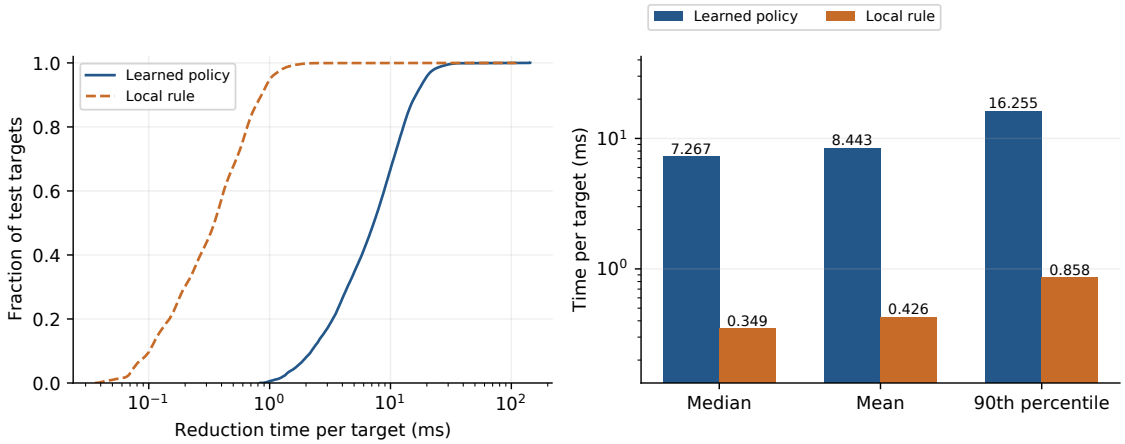


Figure 12: Time per attempted target reduction on one CPU, including unsuccessful attempts. The cumulative distribution and summary statistics include candidate selection and exact row updates.

10.5 Scope of the test results

The learned policy completed more targets than the fixed local rule and had a higher mean measured reduction time. The present implementation therefore provides no speedup over this local rule. The operation-count comparison above measures the cost of the completed paths, while the failure counts identify targets whose supplied equations did not support the sequence selected by the policy. The confidence comparison concerns agreement with a recorded action. A valid alternative can have a different equation identifier or intermediate row, which is why the complete coefficient checks accompany the choice metrics.

The test covers one changing target row reduced with fixed source identities. Most reserved families have the same one-loop box topology with different mass and kinematic conditions, and some lower-sector structures occur in both training and testing. The two-loop result comes from one reserved propagator family. These measurements therefore establish performance within this collection and supplied action rule. Reductions of larger evolving matrices, additional high-loop families and complete Kira jobs require separate measurements.

Citations

[1] Stefan Weinzierl. Introduction to Feynman integrals, 2010. arXiv:1005.1855.

[2] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[3] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[4] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[5] Thomas Gehrmann and Ettore Remiddi. Differential equations for two-loop four-point functions. *Nuclear Physics B*, 580:485–518, 2000. arXiv:hep-ph/9912329.

[6] A. V. Smirnov and A. V. Petukhov. The number of master integrals is finite, 2010. arXiv:1004.4199.

[7] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[8] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[9] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection, 2025. arXiv:2505.20197.

[10] Philipp Kant. Finding linear dependencies in integration-by-parts equations: A Monte Carlo approach. *Computer Physics Communications*, 185:1473–1476, 2014. arXiv:1309.7287.

[11] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[12] Rushil Saraswat. Master-integral bases for sixteen one-loop and two-loop families. Accompanying family definitions, master bases and validation data, 2026.

[13] Rushil Saraswat. Target integrals constructed from master bases with seed and elimination records. Accompanying target integrals, source identities and validation data, 2026.

[14] Rushil Saraswat. Master bases and constructed targets for one thousand integral families. Accompanying family definitions, master bases, target reductions and validation data, 2026.

[15] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.06114.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[18] Rushil Saraswat. Held-out seed-selection predictions, test metrics and figures. Accompanying test probabilities, reference labels, metrics and figures, 2026.

[19] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[20] scikit-learn developers. Average precision score: Definition and treatment of score thresholds. scikit-learn 1.5 documentation, 2024.

[21] Rushil Saraswat. Next-step reduction model and training records for constructed feynman integral targets. Accompanying model, training features, saved parameters and validation records, 2026.

[22] Rushil Saraswat. Frozen reduction-propagation model: Held-out predictions, paths and test results. GitHub test records, 2026. Saved-model hash and input provenance accompany the probability records, exact sampled paths, metrics and figures; accessed 5 October 2026.

[23] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. arXiv:1706.04599; Proceedings of ICML 2017.